

Kubernetes



Exemples – Codes & images docker

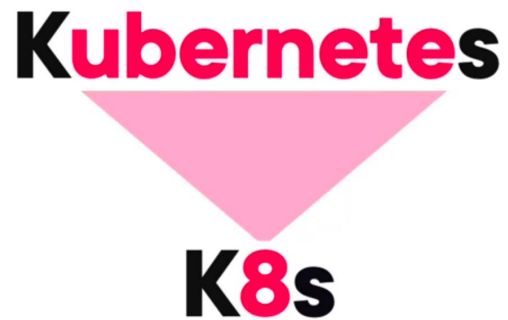


https://github.com/TRIGianni/kubernetes_exemples



<https://hub.docker.com/repositories/heh8mons7000>

Définition



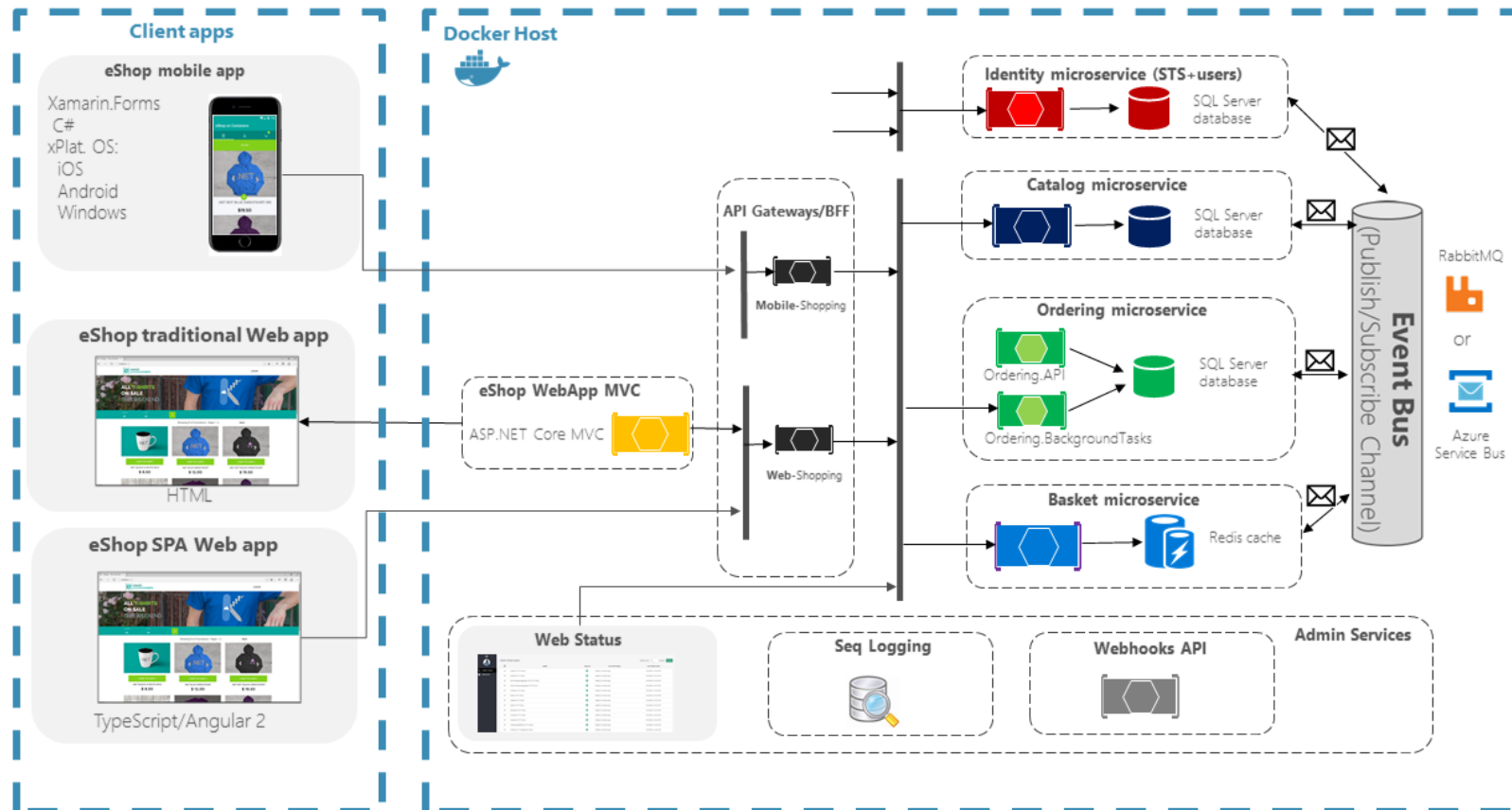
Kubernetes(K8s) est un **orchestrateur d'applications conteneurisées (app microservices)**.

Orchestrateur a pour mission :

- le **déploiement** d'une l'application distribuées
- la **mise à l'échelle** en fonction de la demande (Horizontal scaling)
- le **redémarrage** des conteneurs qui échouent (Self-healing)
- d'effectuer des **prises à jour et des retours en arrière sans temps d'arrêt** (Automated rollouts and rollbacks).

Utilisation

eShopOnContainers reference application (Development environment architecture)



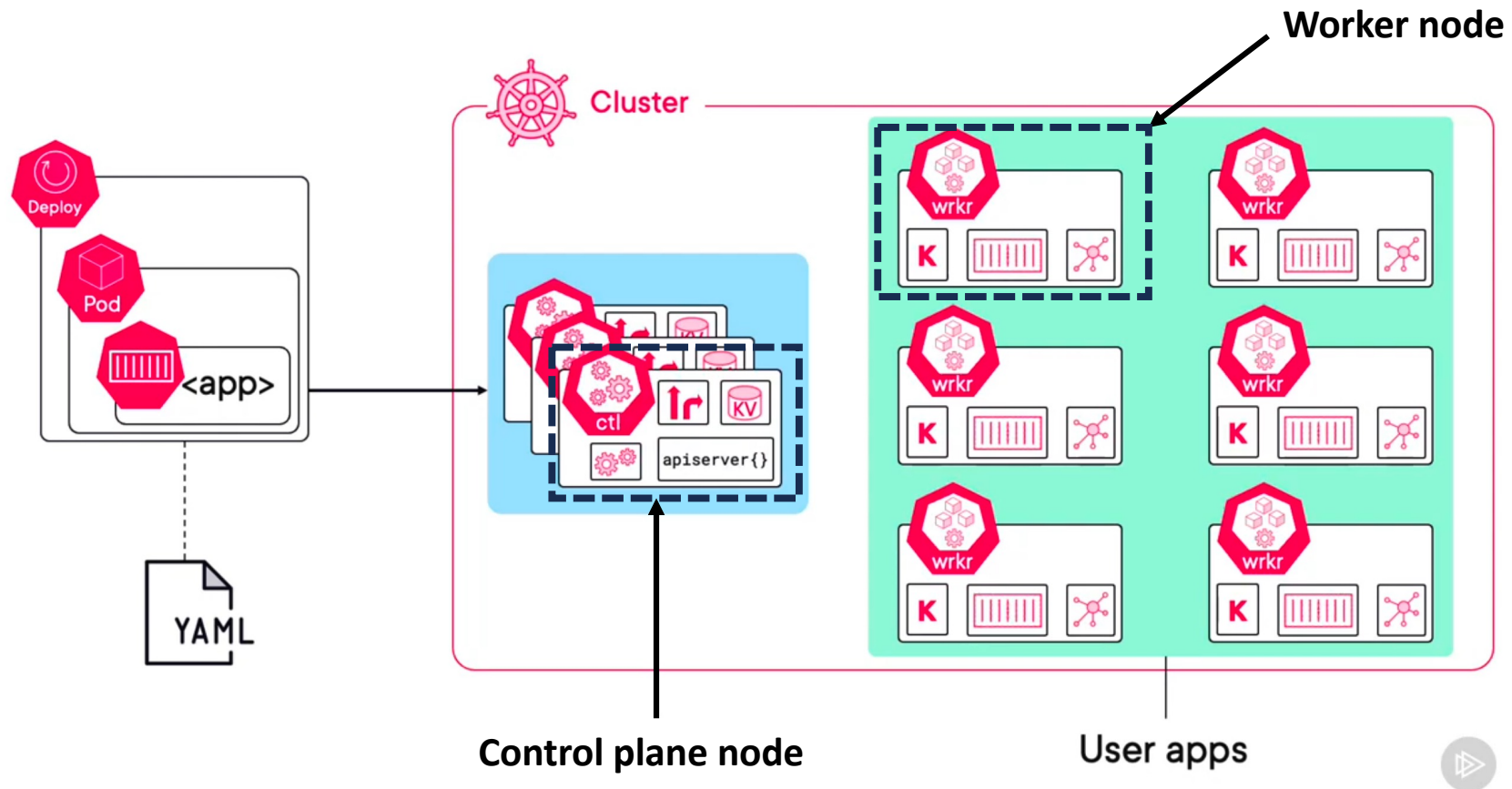
Architecture - Cluster

Cluster : Ensemble de Nodes (machines physiques ou VM).

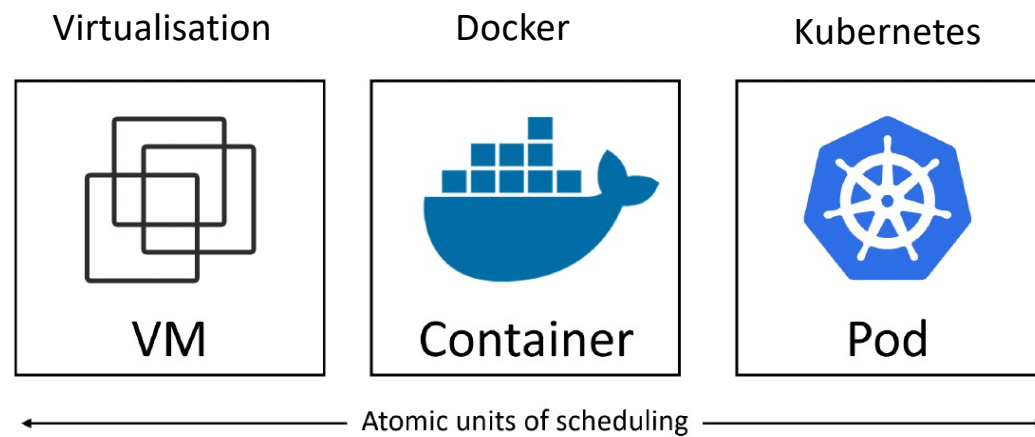
Un **cluster Kubernetes** est constitué de deux types de ressources :

- Le **Control Plane** coordonne le cluster.
- Les **Nodes workers** exécutent des applications conteneurisées.

Architecture – Vue générale



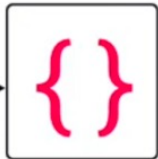
Pod



Pod = unité de déploiement dans Kubernetes

Architecture – Control Plane

Composants :



API server :

- point d'entrée exposant l'API REST de Kubernetes => commande d'administration.
- Consomme YAML/JSON



Scheduler (planificateur) : sélectionne le **nœud (worker)** sur lequel un Pod sera lancé.

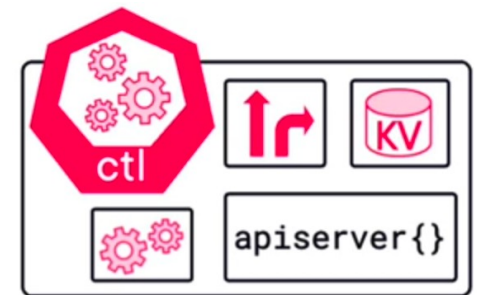


Cluster store(etcd) : base de données clé-valeur (**Key-Value**) qui stocke l'ensemble des données du cluster (état et configuration).

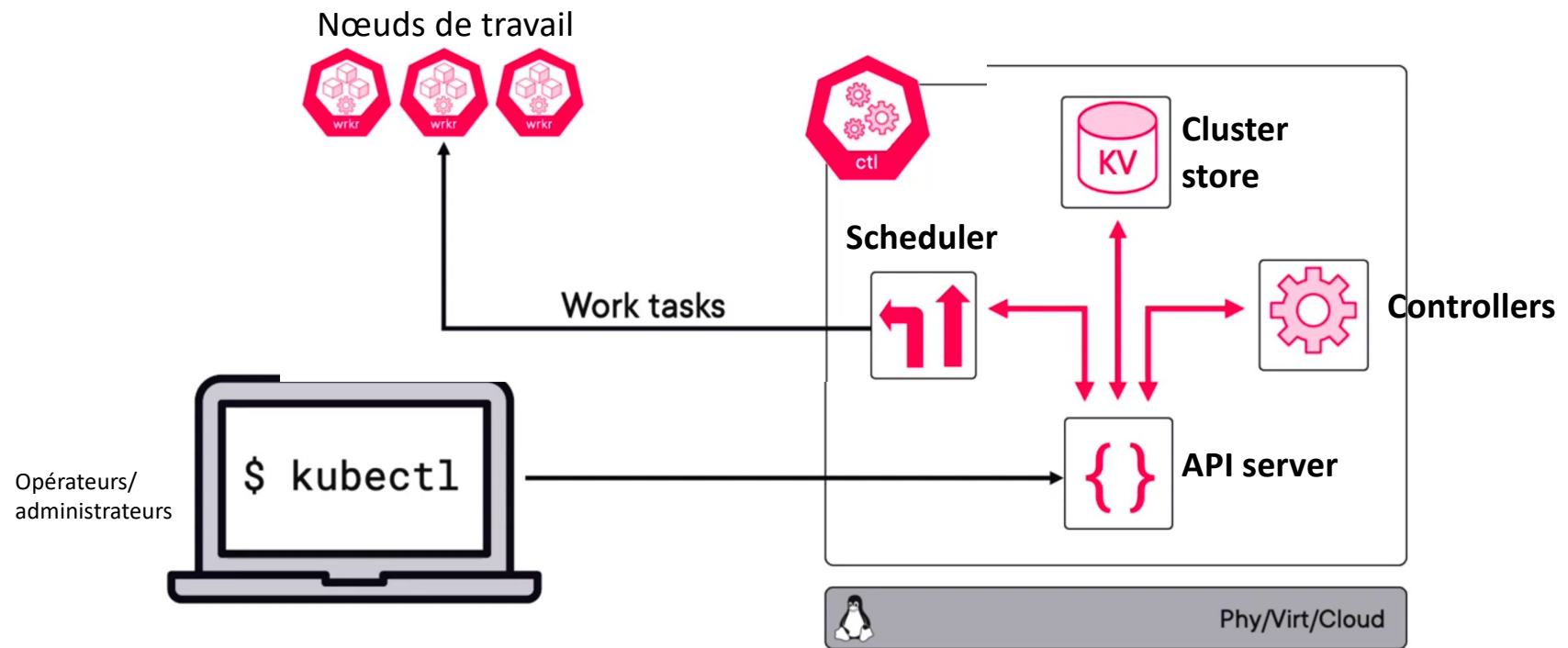


Controllers :

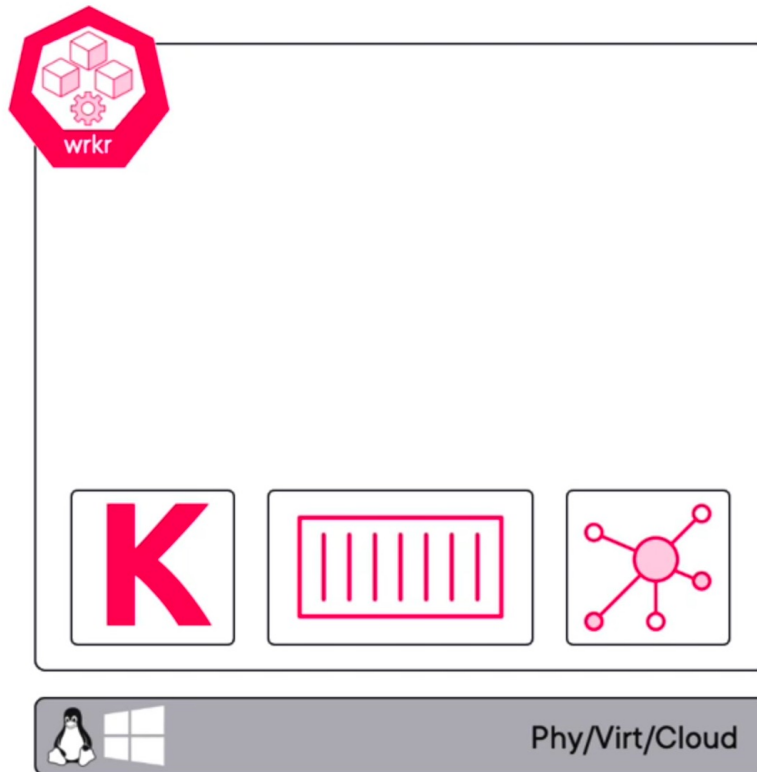
- processus de gestion des différents contrôleurs (node, replication, endpoint, ...)
- Boucles d'observation



Architecture – Control Plane



Architecture – Worker nodes



Kubelet :

- Assure que les containers d'un Pod tournent conformément à la spécification
- Communique avec **Control Plane**



Container runtime :

Environnement d'exécution des containers

- Docker
- Containerd

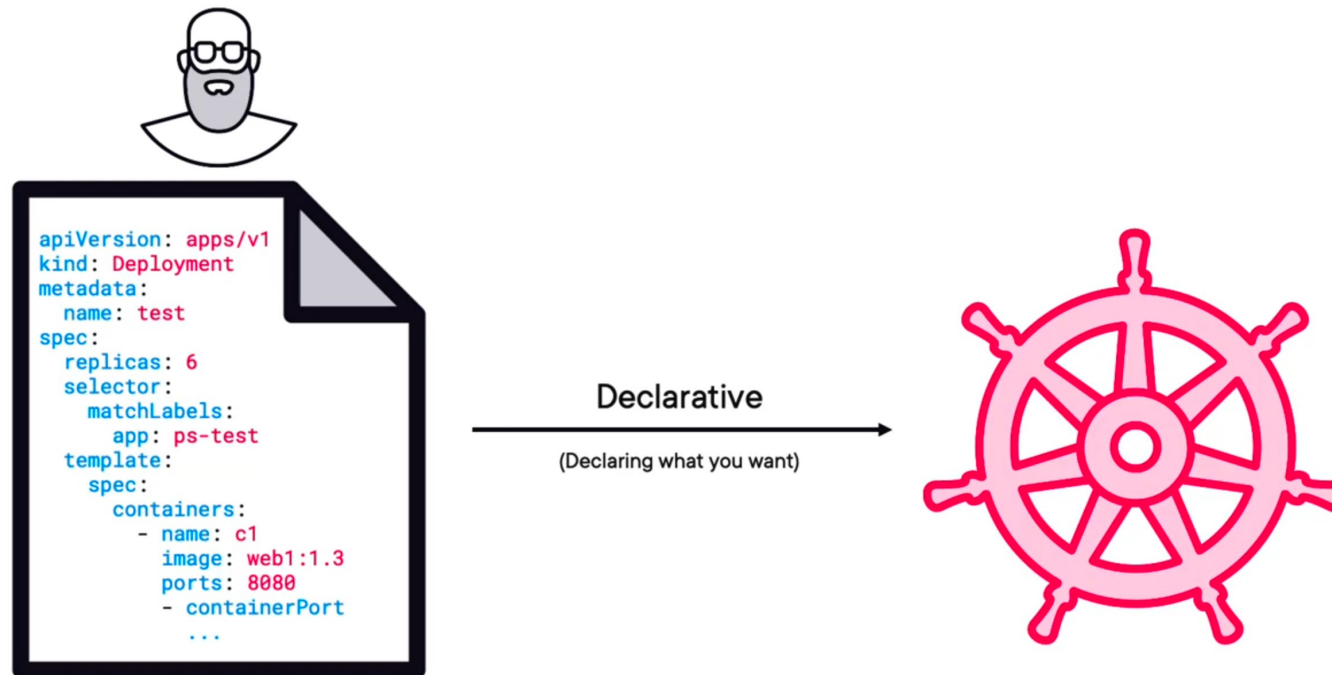


Network Proxy:

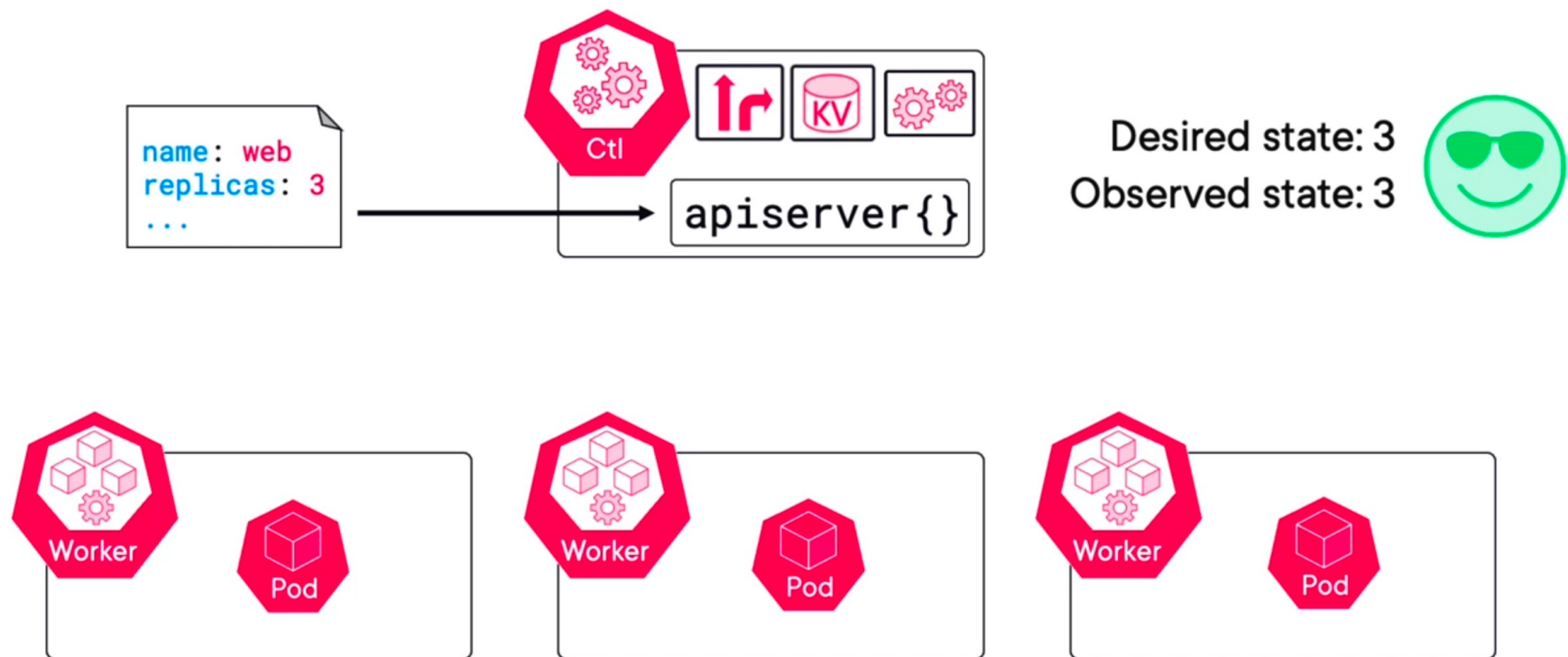
Exposition des services vers l'extérieur.

Modèle déclaratif et état désiré

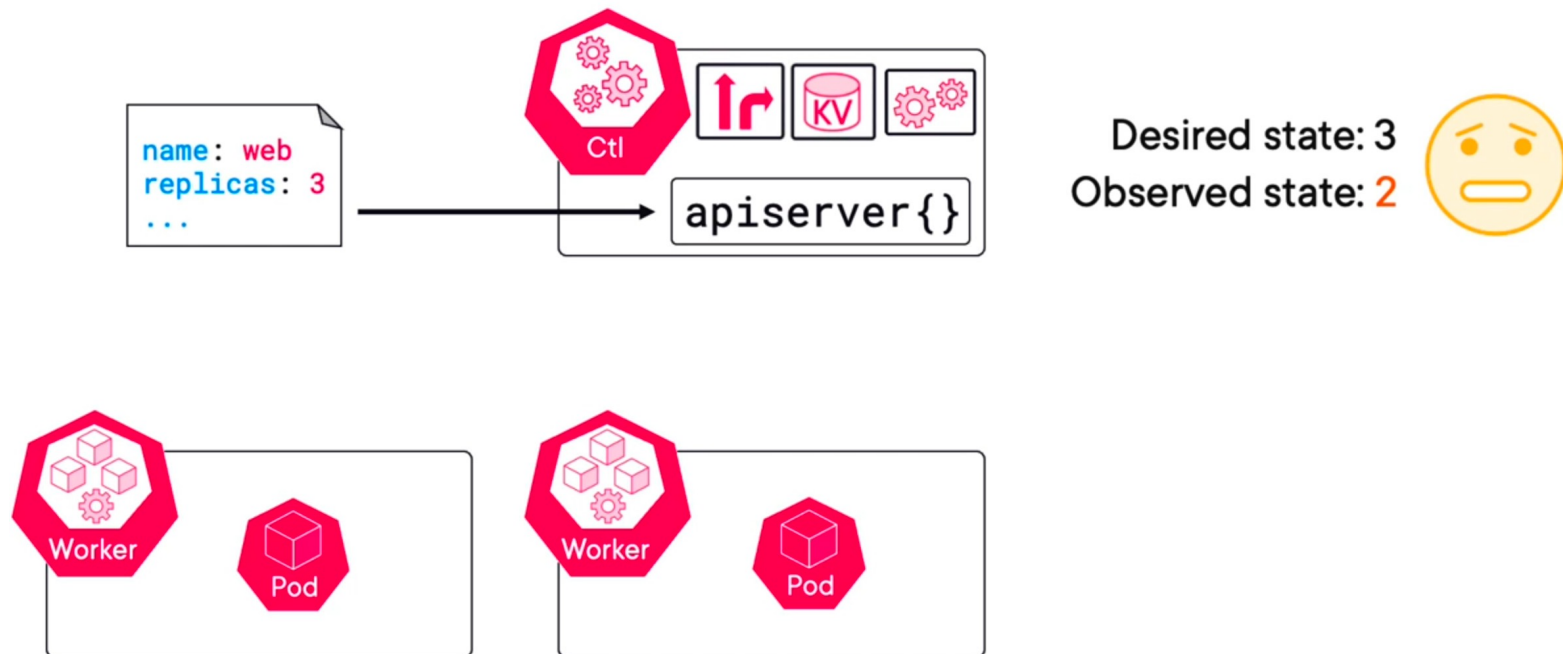
Décrire ce que **vous voulez** (état souhaité) dans un fichier de configuration.



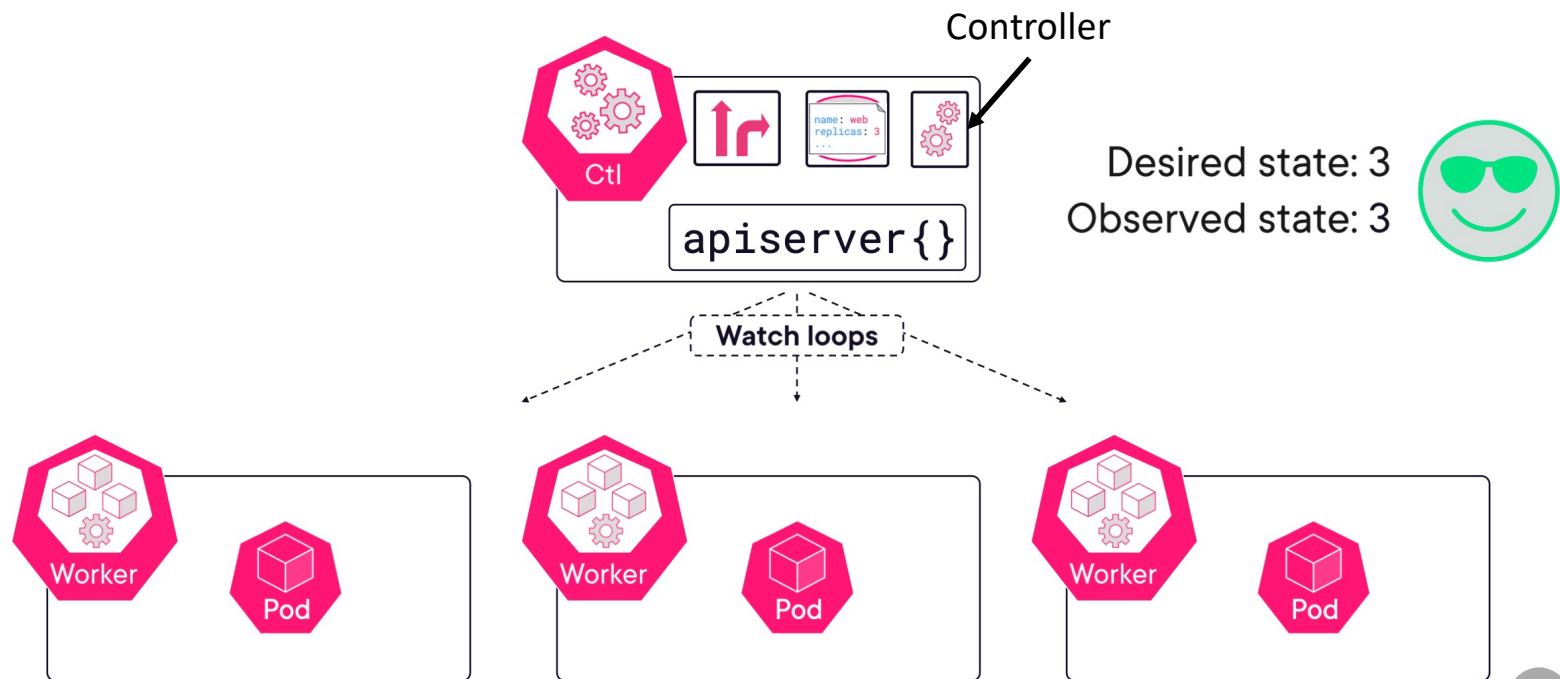
Modèle déclaratif et état désiré



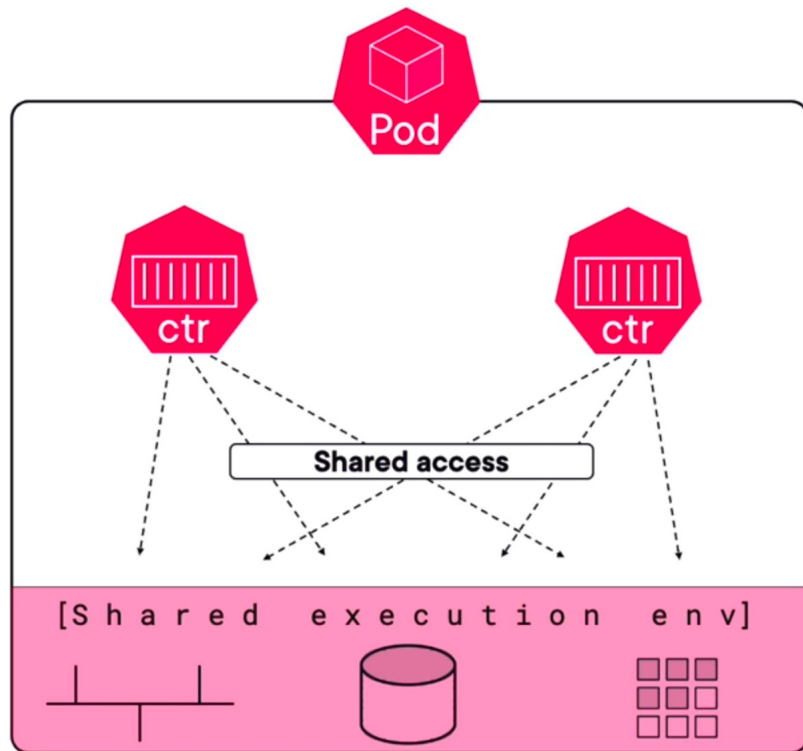
Modèle déclaratif et état désiré



Modèle déclaratif et état désiré



Architecture - Pod



Les **pods** sont l'unité atomique de la plate-forme Kubernetes.

Un **pod** représente un groupe d'un ou plusieurs **conteneurs** d'application (tels que Docker) qui partagent une **IP** et des **volumes**

Environnement pour des containers.

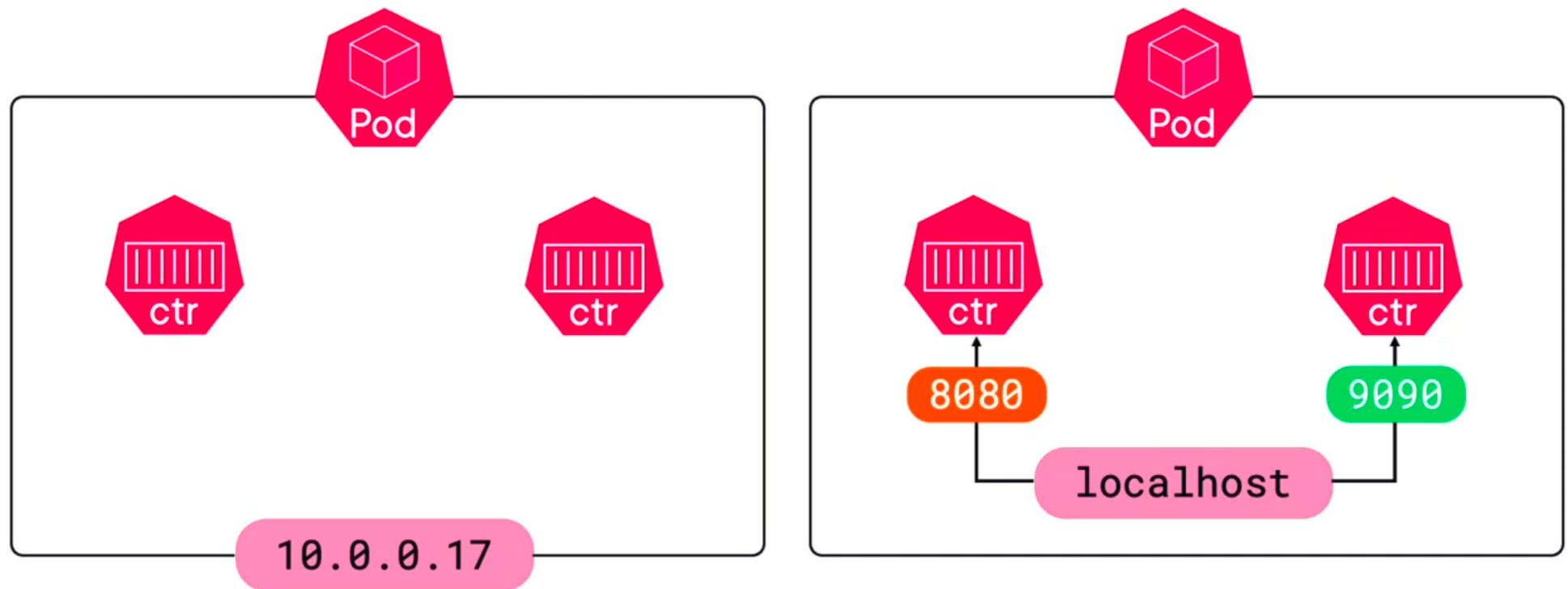
Découper l'application en Pod (APIs, base de données, ...)

Pod IP, mémoire, volume partagé au travers les containers.

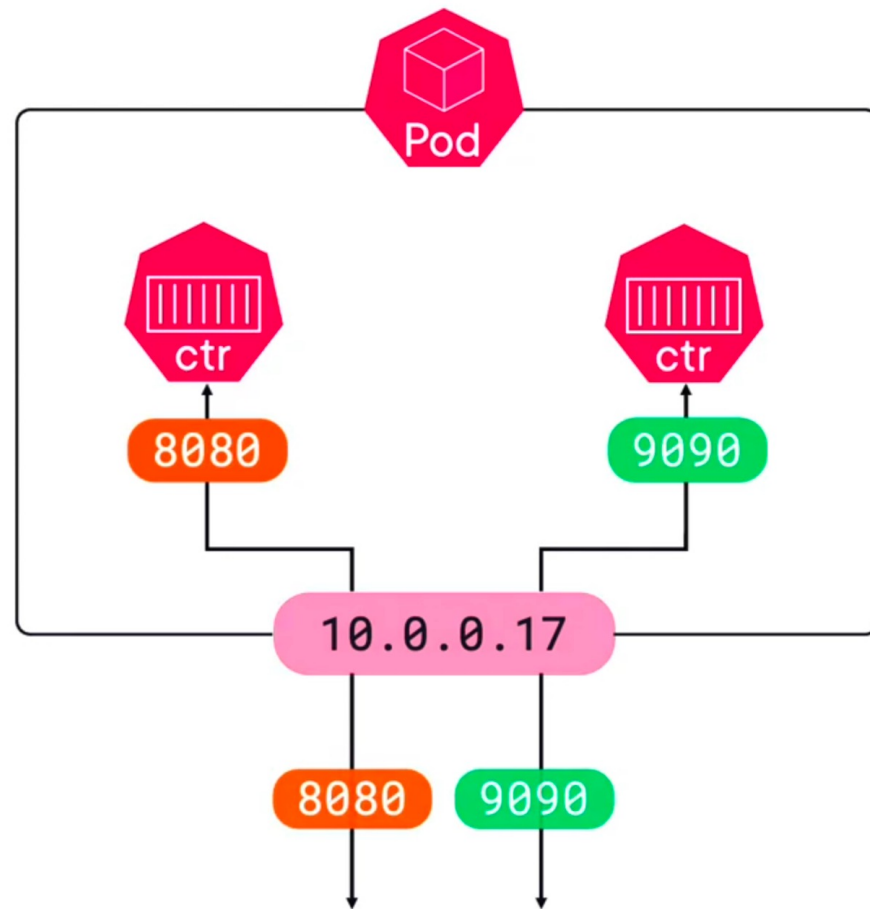
Mise à échelle horizontale via le nombre de réplica d'un Pod

Les pods vivent et meurent.

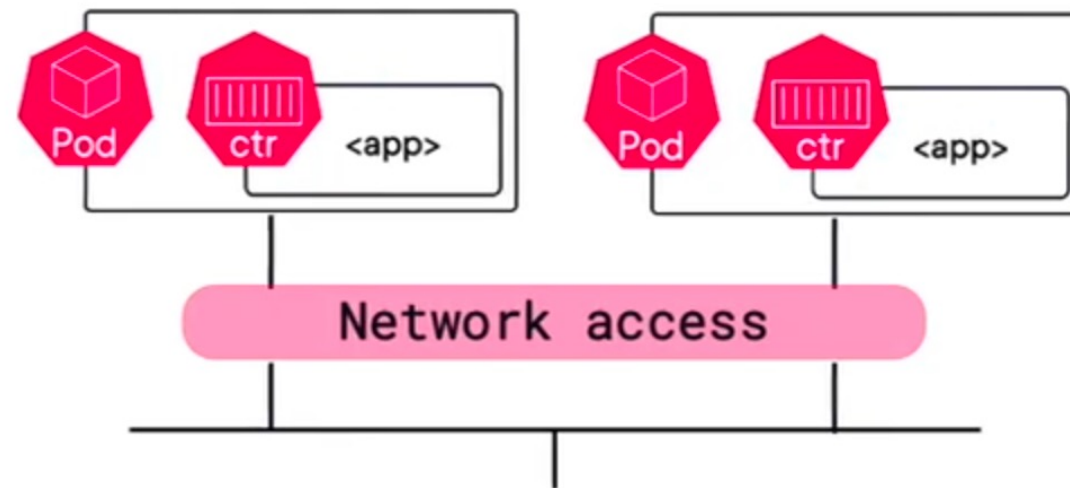
Architecture - Pod



Architecture - Pod



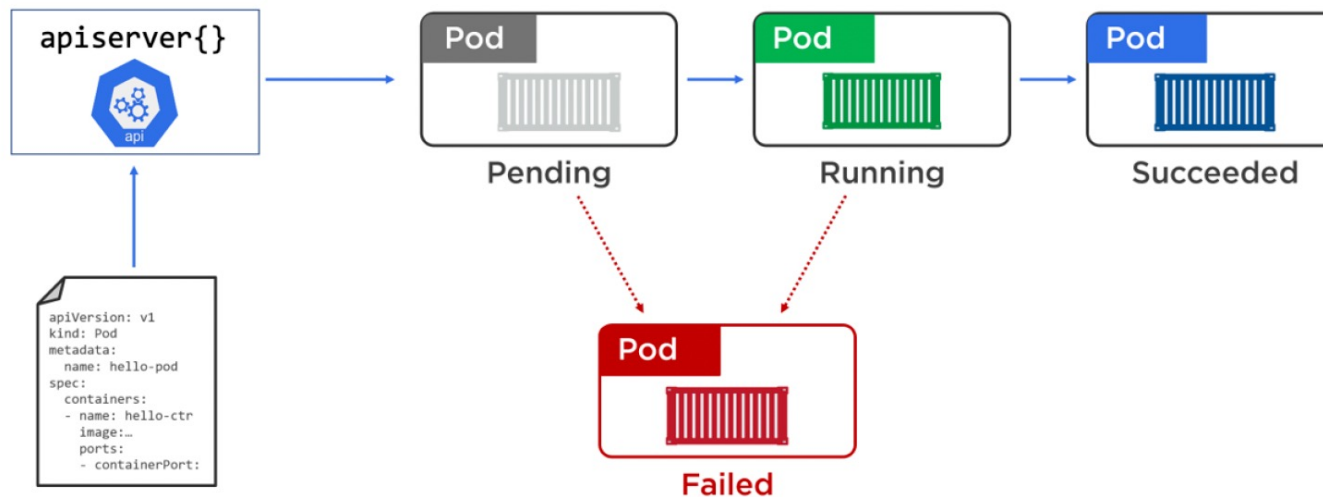
Architecture - Pod



Ressources/objets – Cycle de vie Pods

Pending : télécharge les images et démarre les conteneurs.

Running : Le pod a été affecté à un Node et tous les conteneurs ont été créés. Au moins un conteneur est en cours d'exécution.



Ressources/objets K8s – Catégories

- Gestion des applications lancées sur le cluster (Pod, Deployment, ...)
- Discovery et load balacing (Service, ...)
- Configuration des applications (ConfigMap, Secret, ...)
- Stokage (PersistentVolume, ...)
- Configuration du cluster et metadata (Namespace, ...)

Ressources/objets Kubernetes

Fichier de configuration – Décrire les ressources (manifest)

Différentes catégories de ressources = même structure de base du manifest

apiVersion : spécifie la version de l'API Kubernetes

kind : définit le type de ressource créée ou modifiée. Les types les plus courants sont « Deployment », « Service », « Pod », « ConfigMap », et « Ingress ».

metadata : contient des informations essentielles sur la ressource, telles que son nom, ses étiquettes et ses annotations.

spec : décrit l'**état désiré** de la ressource. Il décrit les détails de la configuration et le comportement de la ressource. La structure et le contenu de ce champ varient en fonction du type de ressource.

Fichier de configuration – Décrire les ressources (manifest)

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: www
      image: nginx:1.12.2
```

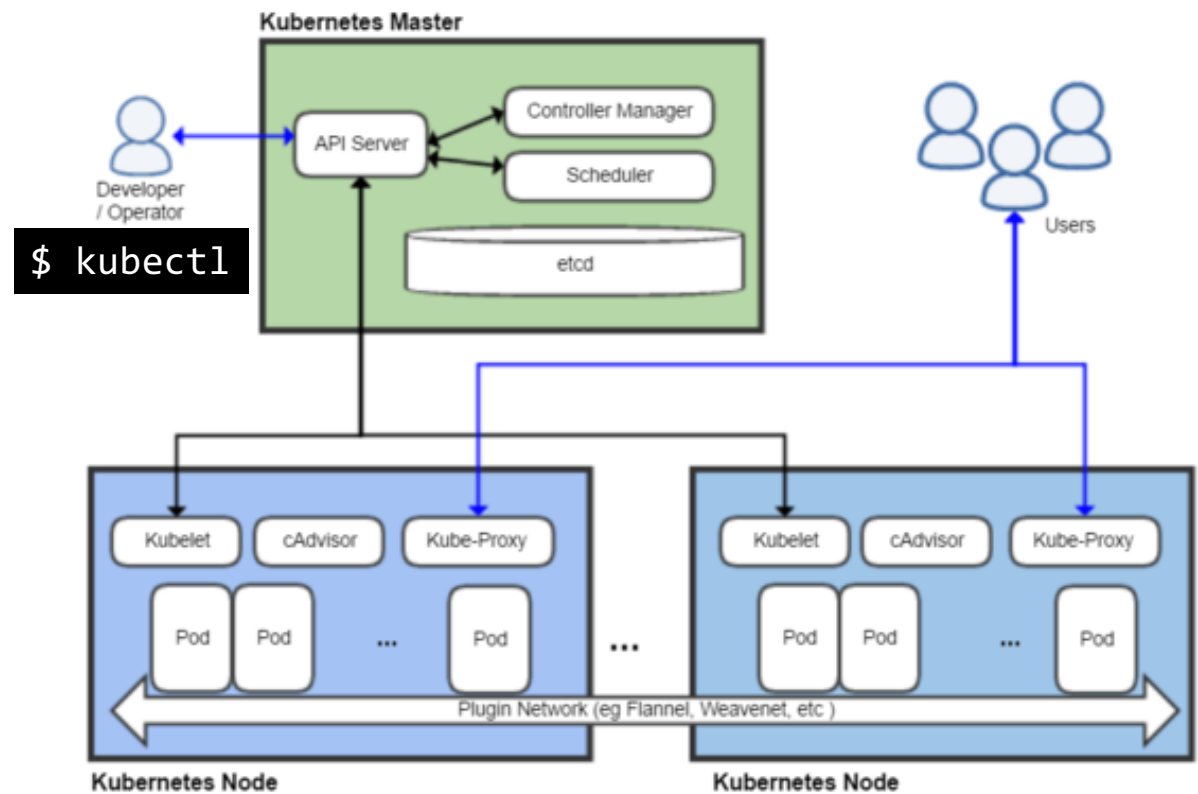
```
apiVersion: v1
kind: Service
metadata:
  name: vote-service
spec:
  selector:
    app: vote
  ports:
    - port: 80
      targetPort: 80
```

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: www-domain
spec:
  rules:
    - host: www.example.com
      http:
        paths:
          - backend:
              serviceName: www
              servicePort: 80
```

Outil CLI – Kubectl

Kubectl est outil :

- pour administrer un cluster K8s
- en ligne de commande (CLI)
- Qui communique avec le Control Panel



Outil CLI – Kubectl

```
kubectl [command] [TYPE] [NAME] [flags]
```

- *command*:
Spécifie **l'opération** à effectuer sur une ou plusieurs ressources, par exemple *apply*, *create*, *get*, *describe* et *delete*.
- *TYPE*:
Spécifie le **type de ressource**. Les types de ressources ne sont pas sensibles à la casse et vous pouvez spécifier les formes singulières, plurielles ou abrégées. Par exemple, les commandes suivantes produisent le même résultat :

```
kubectl get pod pod1  
kubectl get pods pod1  
kubectl get po pod1
```

- *NAME*: Spécifie le nom de la ressource. **Les noms sont sensibles à la casse.**

Outil CLI – Kubectl

```
kubectl [command] [TYPE] [NAME] [flags]
```

- *Flags* : Spécifie des drapeaux facultatifs. Par exemple, vous pouvez utiliser les options -s ou --server pour spécifier l'adresse et le port du serveur API Kubernetes.

Outil CLI – Kubectl

```
kubectl version  
kubectl cluster-info  
kubectl get all  
kubectl run [container-name]  
  --image=[image-name]  
kubectl port-forward [pod] [ports]  
kubectl expose ...  
kubectl create [resource]  
kubectl apply [resource]
```

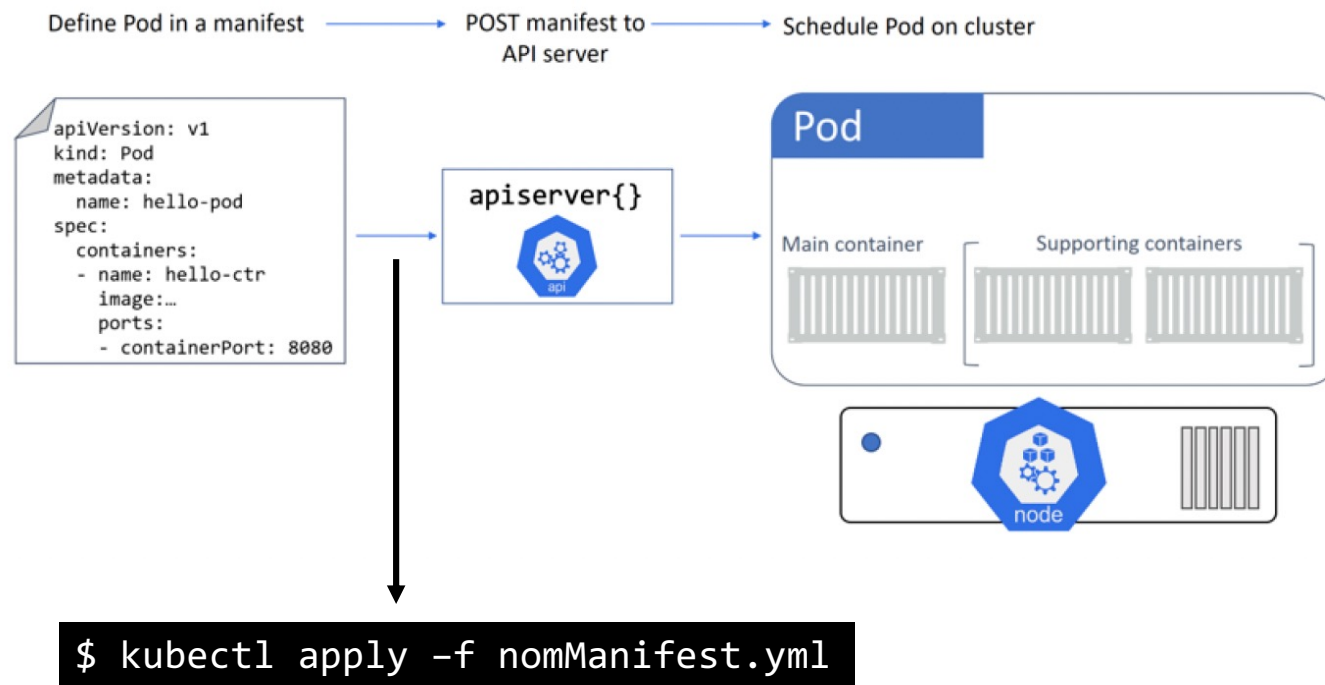
- ◀ Check Kubernetes version
- ◀ View cluster information
- ◀ Retrieve information about Kubernetes Pods, Deployments, Services, and more
- ◀ Simple way to create a Deployment for a Pod
- ◀ Forward a port to allow external access
- ◀ Expose a port for a Deployment/Pod
- ◀ Create a resource
- ◀ Create or modify a resource

Outil CLI – Kubectl

Commande

delete	<code>kubectl delete (-f FILENAME TYPE [NAME /NAME -l label --all]) [flags]</code>	Delete resources either from a file, stdin, or specifying label selectors, names, resource selectors, or resources.
describe	<code>kubectl describe (-f FILENAME TYPE [NAME_PREFIX /NAME -l label]) [flags]</code>	Display the detailed state of one or more resources.
get	<code>kubectl get (-f FILENAME TYPE [NAME /NAME -l label]) [--watch] [--sort-by=FIELD] [[-o --output]=OUTPUT_FORMAT] [flags]</code>	List one or more resources.
exec	<code>kubectl exec POD [-c CONTAINER] [-i] [-t] [flags] [-- COMMAND [args...]]</code>	Execute a command against a container in a pod.

Ressources/objets – Déployer un Pod



Pod – Fichier de configuration (exemple)

```
apiVersion: v1
kind: Pod
metadata:
  name: www-pod
spec:
  containers:
    - name: www-ctr
      image: heh8mons7000/app_web:1.0
      ports:
        - containerPort: 5000
```

<https://www.yamllint.com/>

Pod – Commandes utiles

```
# List only Pods
```

```
kubectl get pods
```

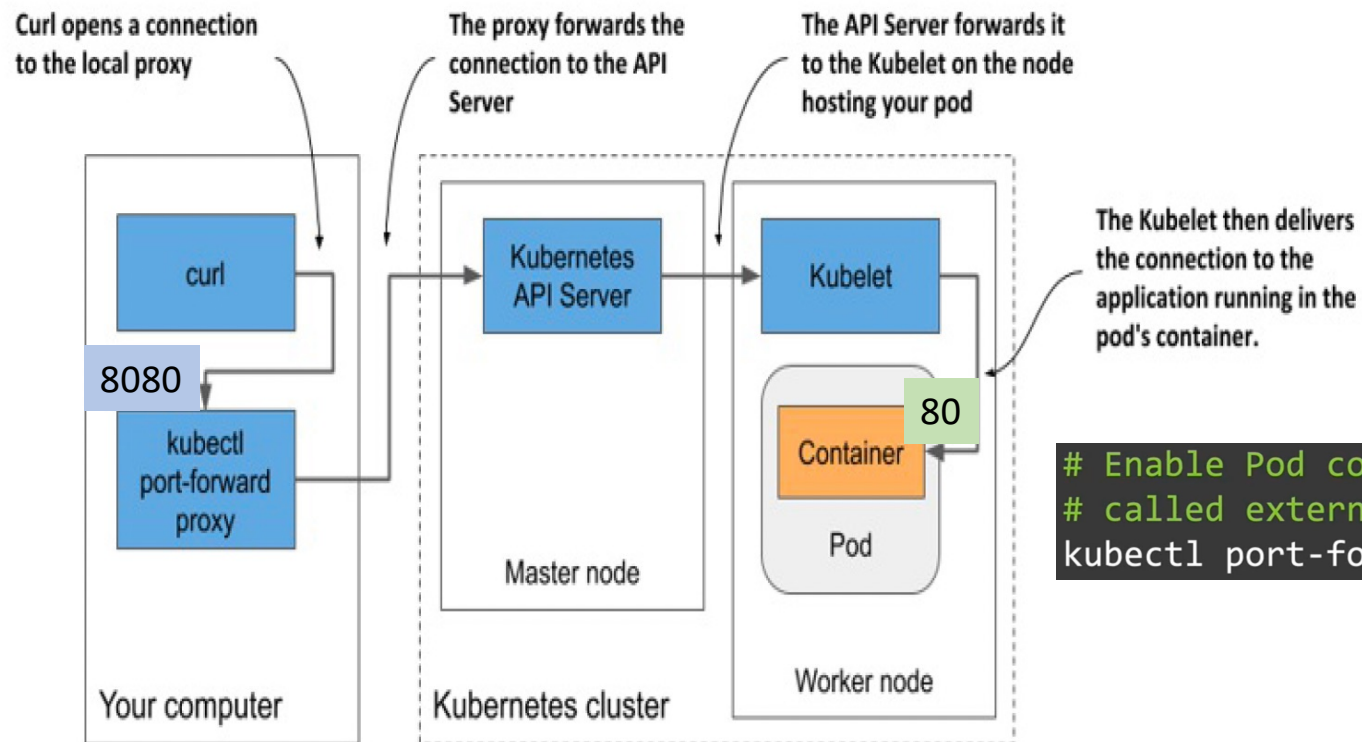
```
# List all resources
```

```
kubectl get all
```

```
# Will cause pod to be recreated
```

```
kubectl delete pod [name-of-pod]
```

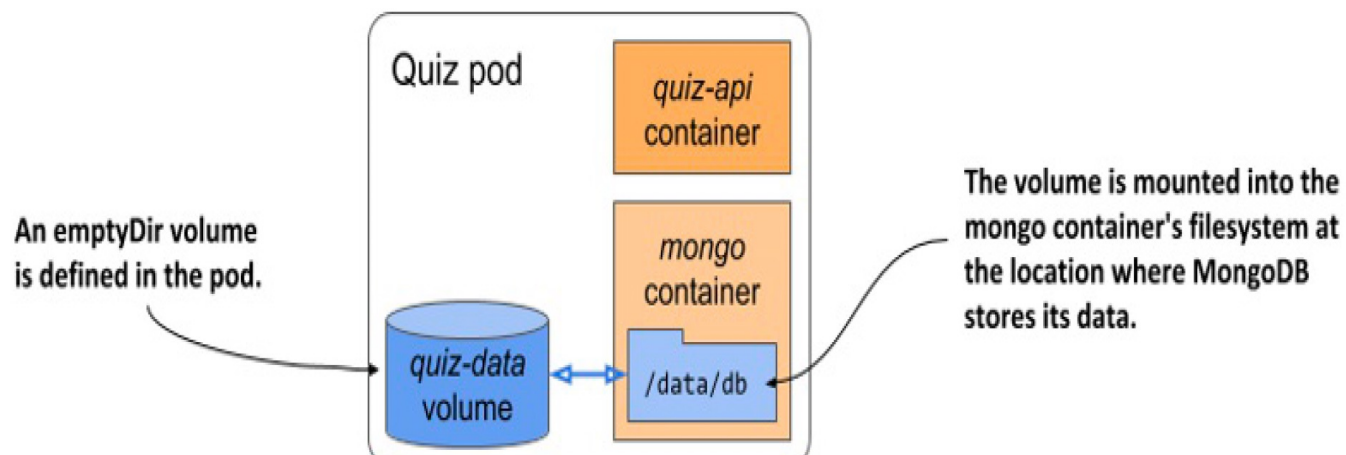
Pod – Redirection de port



```
# Enable Pod container to be  
# called externally  
kubectl port-forward [name-of-pod] 8080:80
```

Pod – Volumes - EmptyDir

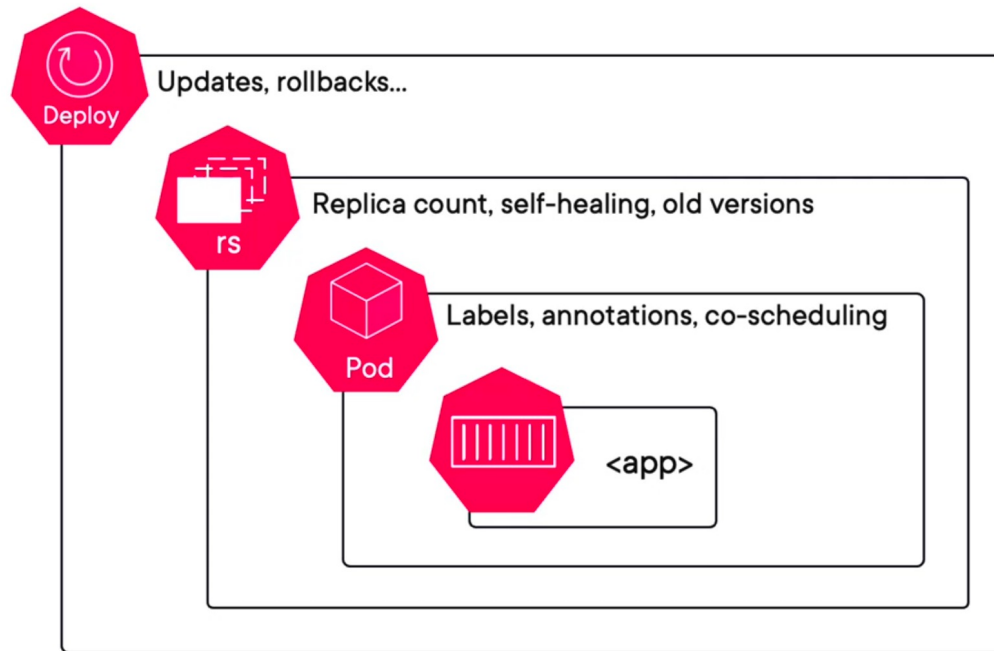
emptyDir - Un simple répertoire qui permet au pod de stocker des données pendant la durée de son cycle de vie.



Pod – Volumes - EmptyDir

```
apiVersion: v1
kind: Pod
metadata:
  name: quiz
spec:
  volumes:      #A
  - name: quiz-data      #A
    emptyDir: {}      #A
  containers:
  - name: quiz-api
    image: luksa/quiz-api:0.1
    ports:
    - name: http
      containerPort: 8080
  - name: mongo
    image: mongo
    volumeMounts:      #B
    - name: quiz-data      #B
      mountPath: /data/db      #B
```

Ressources/objets – Deployment



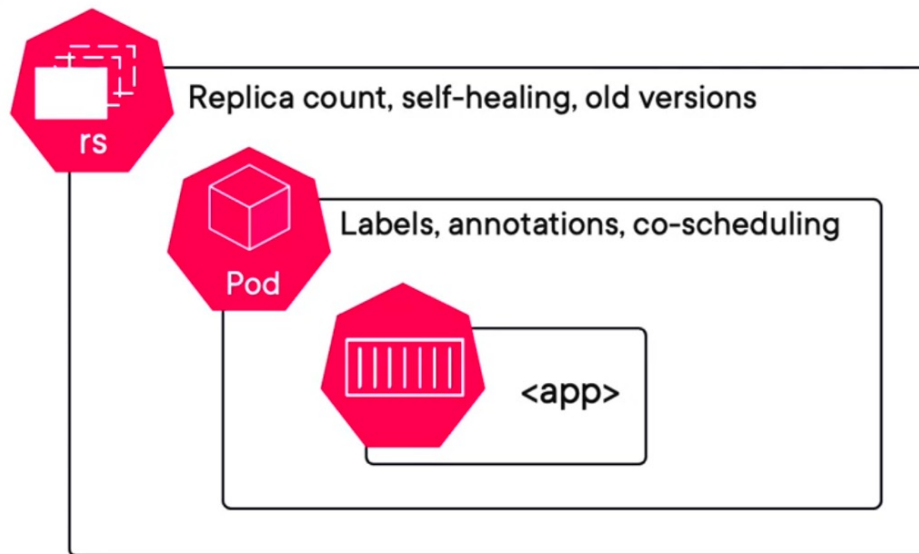
Un **Deployment** gère des **ReplicaSets**.

Un **ReplicaSet (rs)** gère un ensemble de **Pods** de même spécification.

Un **Deployment** définit un état souhaité

- Spécification d'un Pod et du nombre de réplicas voulu.

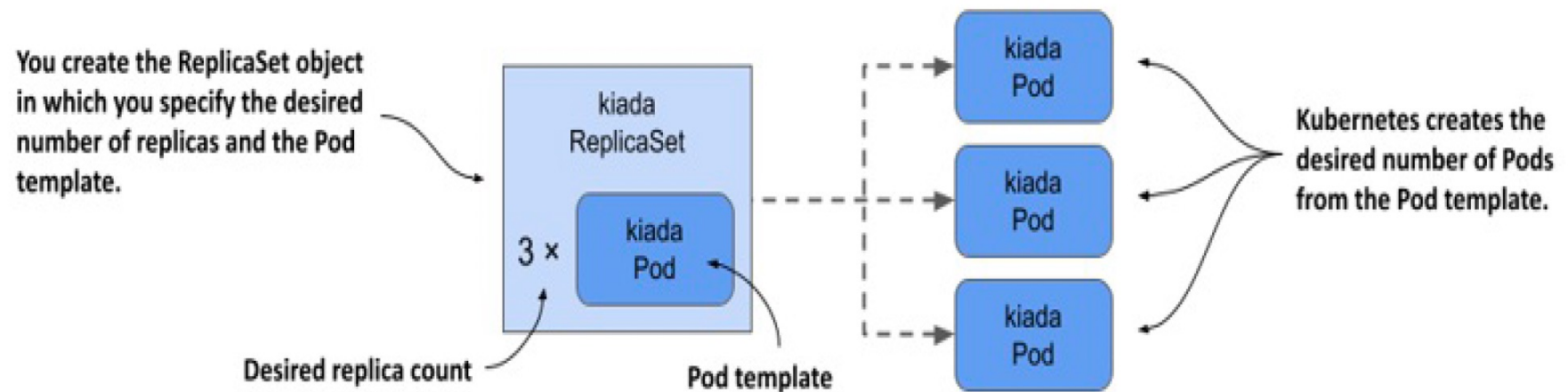
ReplicaSet– Self-healing and scalability



Si un Pod géré par un replicatSet tombe en panne, il sera remplacé – **Self-healing**

Si un Pod géré par un ReplicatSet voit sa charge augmenter, vous pouvez facilement ajouter d'autres Pods pour faire face à la charge - **scalability**

ReplicaSet– Self-healing and scalability



Concepts – Etats et modèle déclaratif

Trois concepts

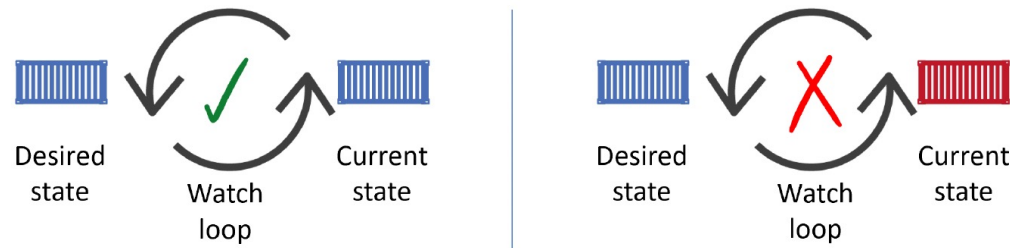
- État souhaité
- État courant (parfois appelé état actuel ou état observé)
- Modèle déclaratif

L'état souhaité est ce que vous voulez.

L'état courant est ce que vous avez.

Le modèle déclaratif est un moyen d'indiquer à Kubernetes l'état souhaité, sans avoir à entrer dans les détails de sa mise en œuvre. Vous laissez le comment à Kubernetes

Concepts – Etats et modèle déclaratif

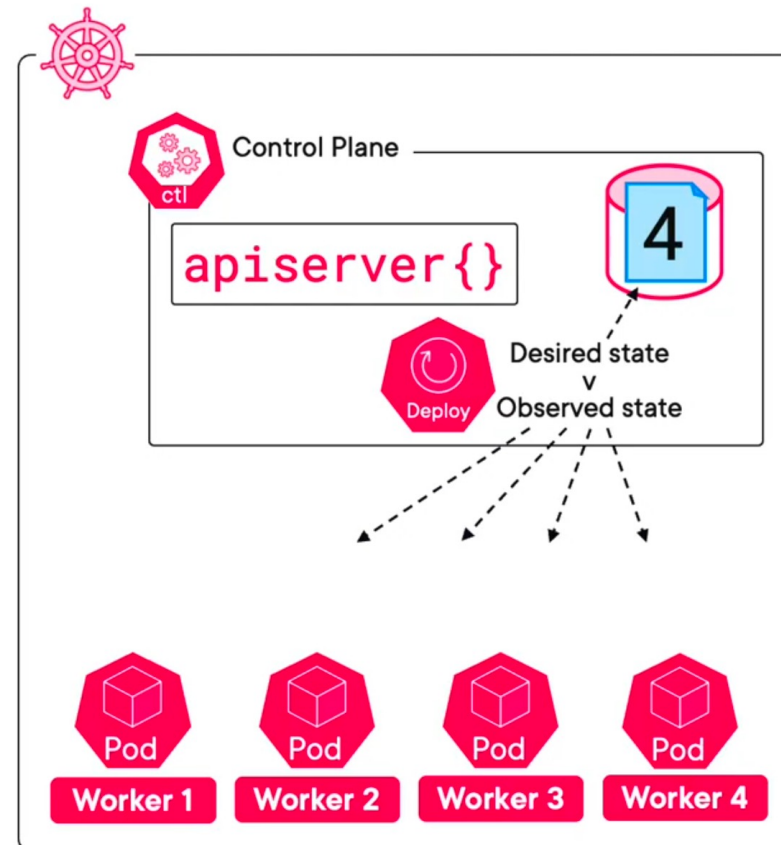


La nature déclarative de Kubernetes met en jeu deux concepts interdépendants : celui de l'état courant et celui de l'état désiré.

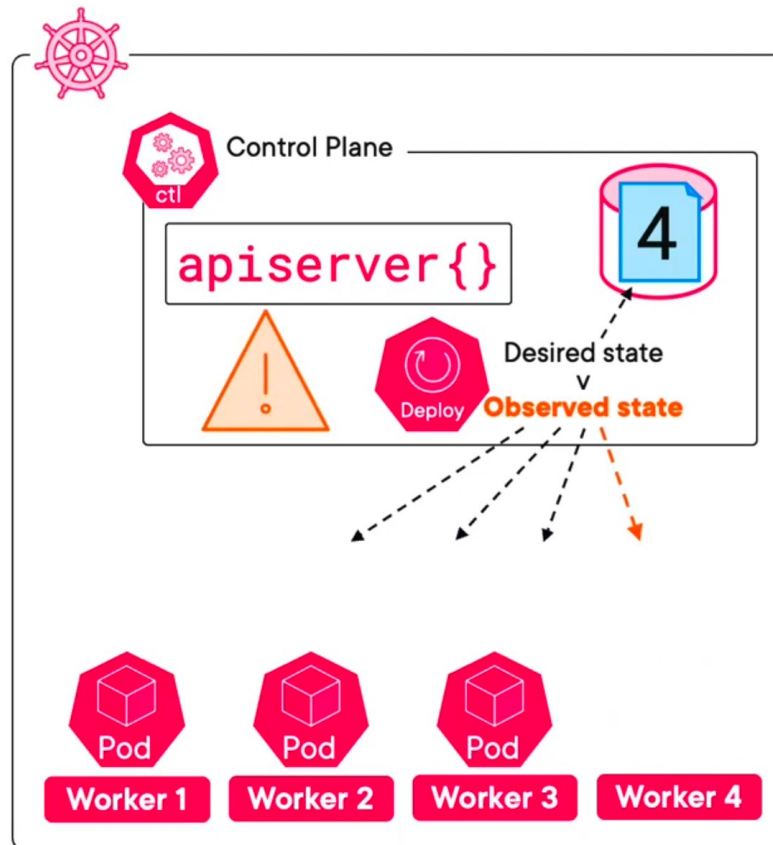
La tâche de Kubernetes est de réaliser la réconciliation de ces deux états en faisant converger l'état courant vers l'état désiré. Ceci est fait via ce qu'on appelle des boucles de réconciliation.

Par exemple, les **ReplicaSets** mettent en œuvre une boucle de réconciliation en arrière-plan qui vérifie constamment si le bon nombre de répliques Pod est présent sur le cluster. S'il n'y en a pas assez, il en ajoute. S'il y en a trop, il en supprime certains.

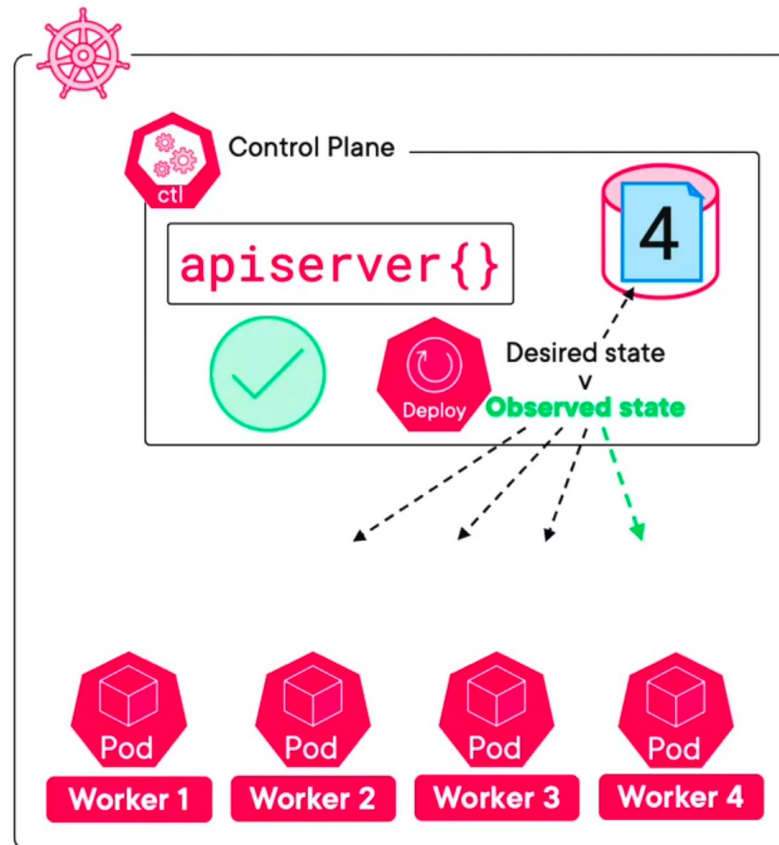
Ressources/objets – Deployment



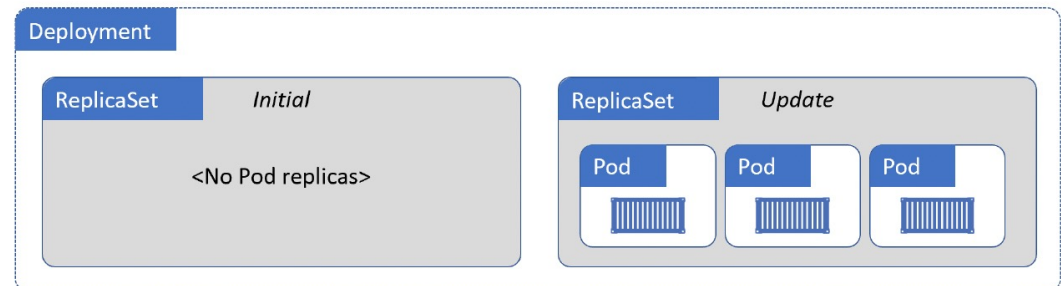
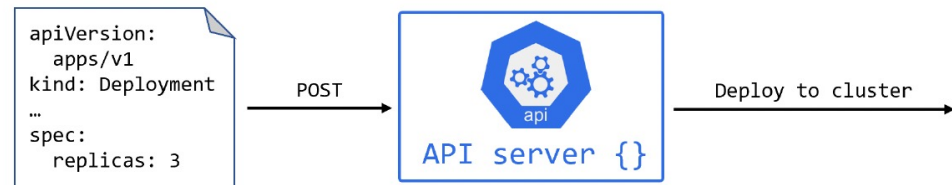
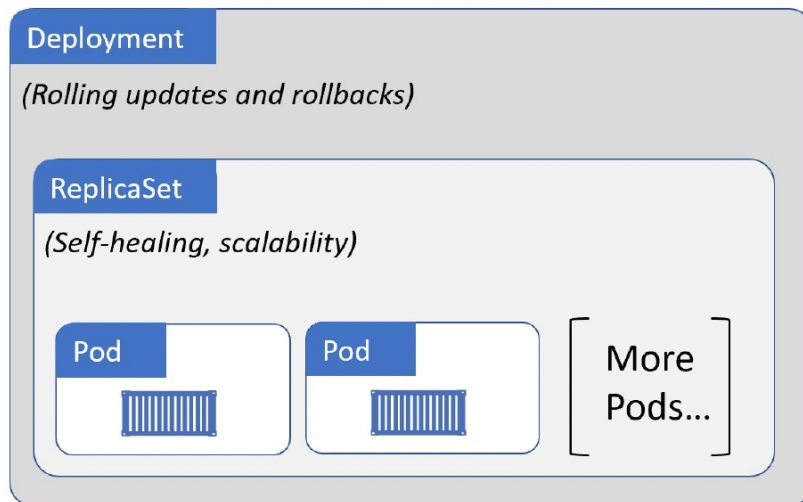
Ressources/objets – Deployment



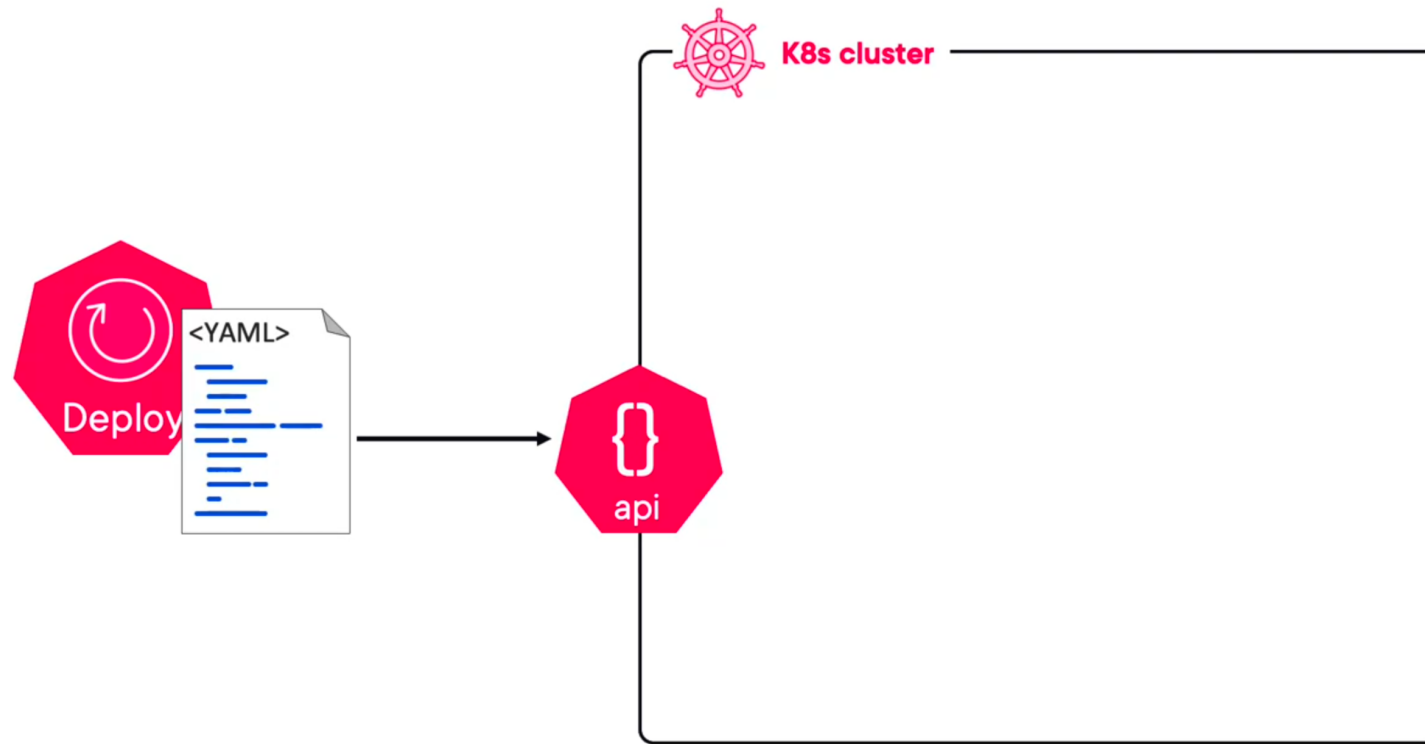
Ressources/objets – Deployment



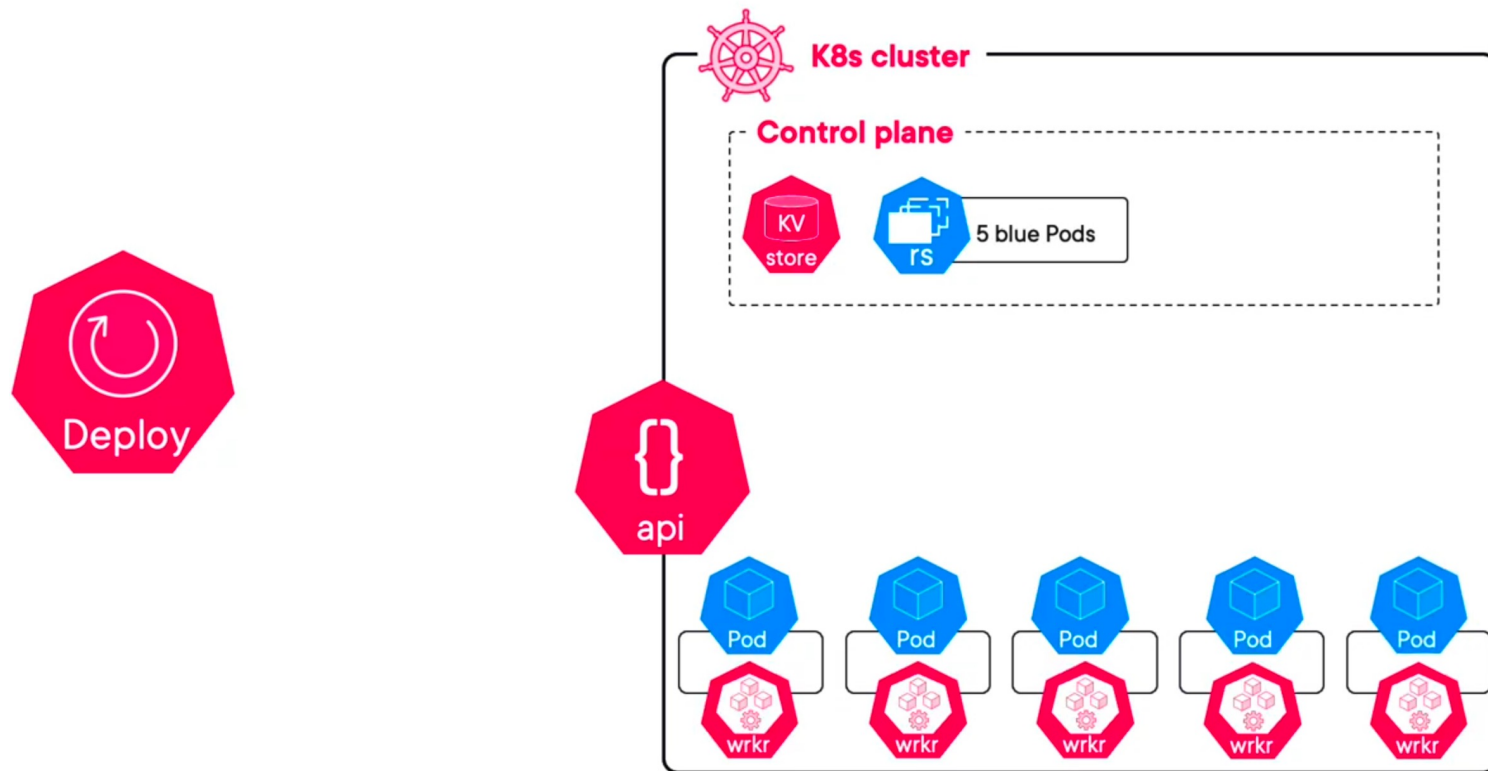
Deployment – Mises à jour en continu (Rolling update)



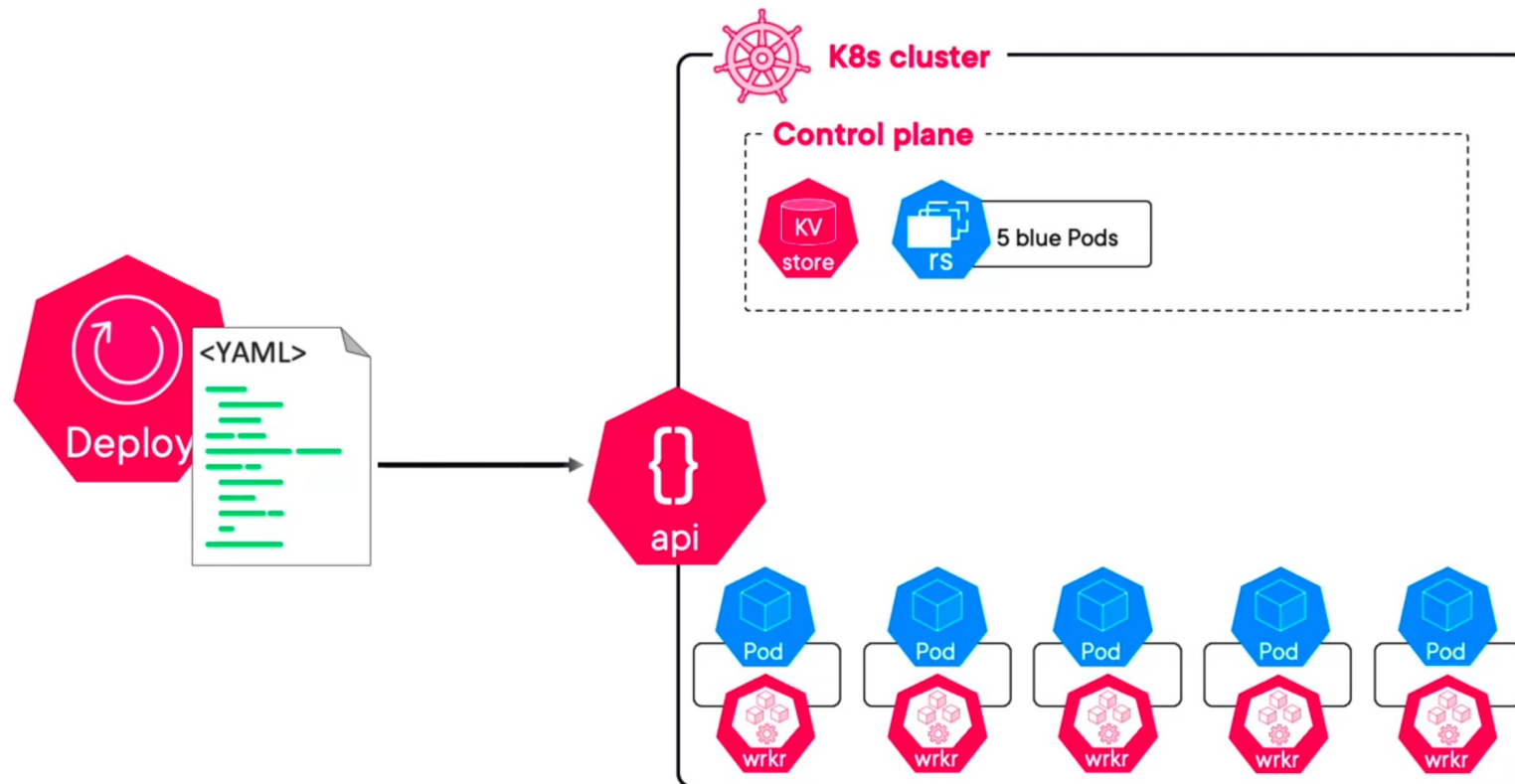
Deployment – Mises à jour en continu (Rolling update)



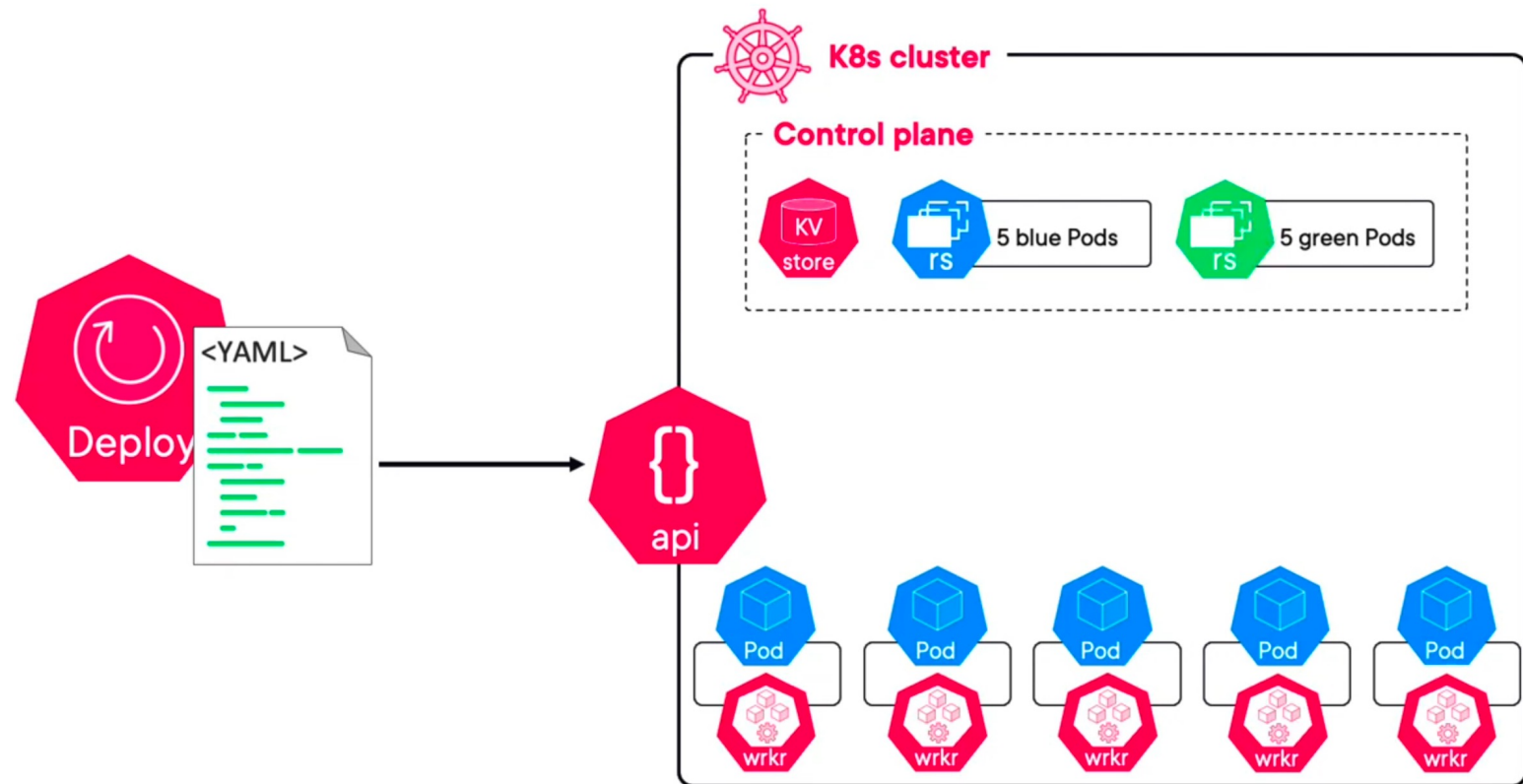
Deployment – Mises à jour en continu (Rolling update)



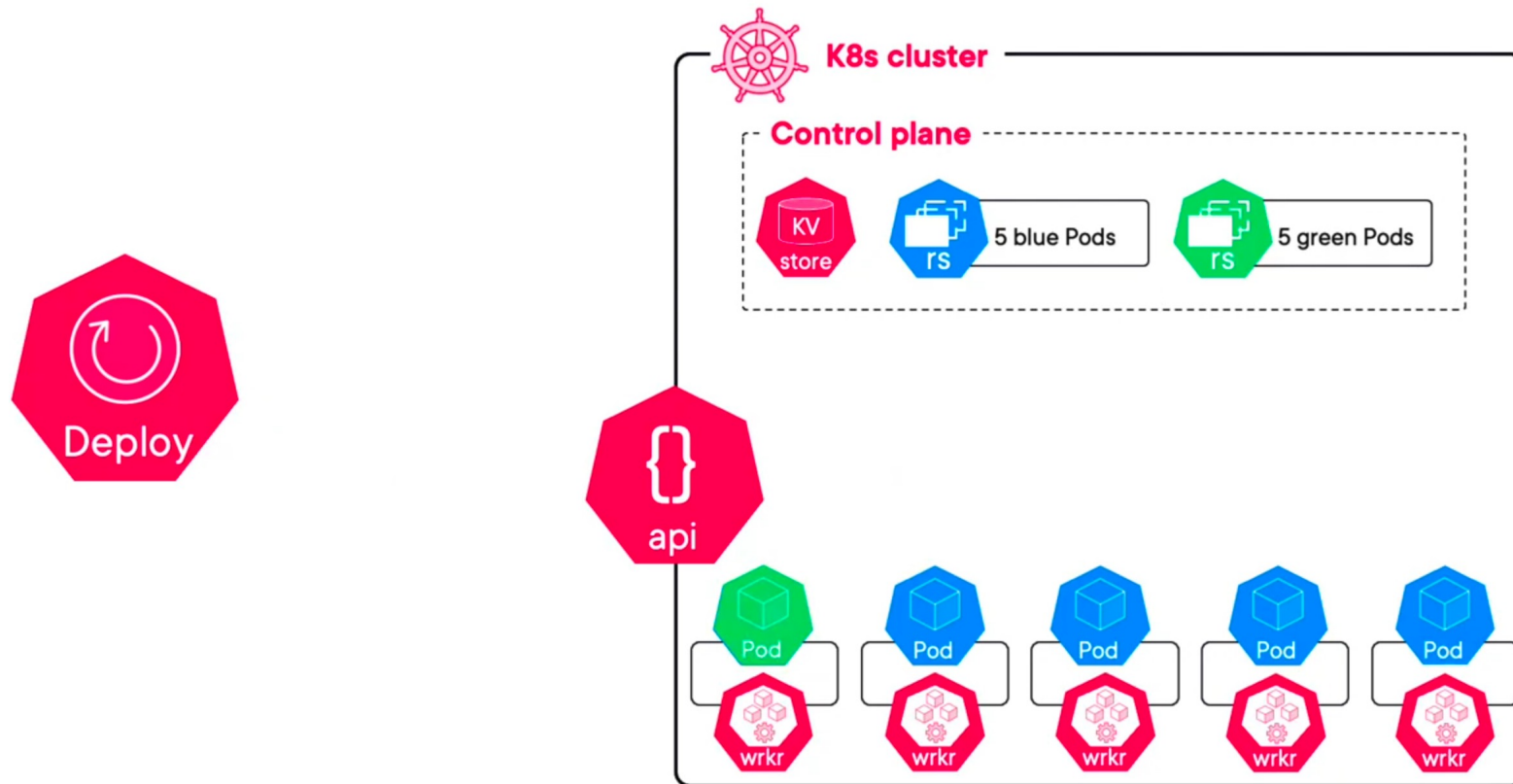
Deployment – Mises à jour en continu (Rolling update)



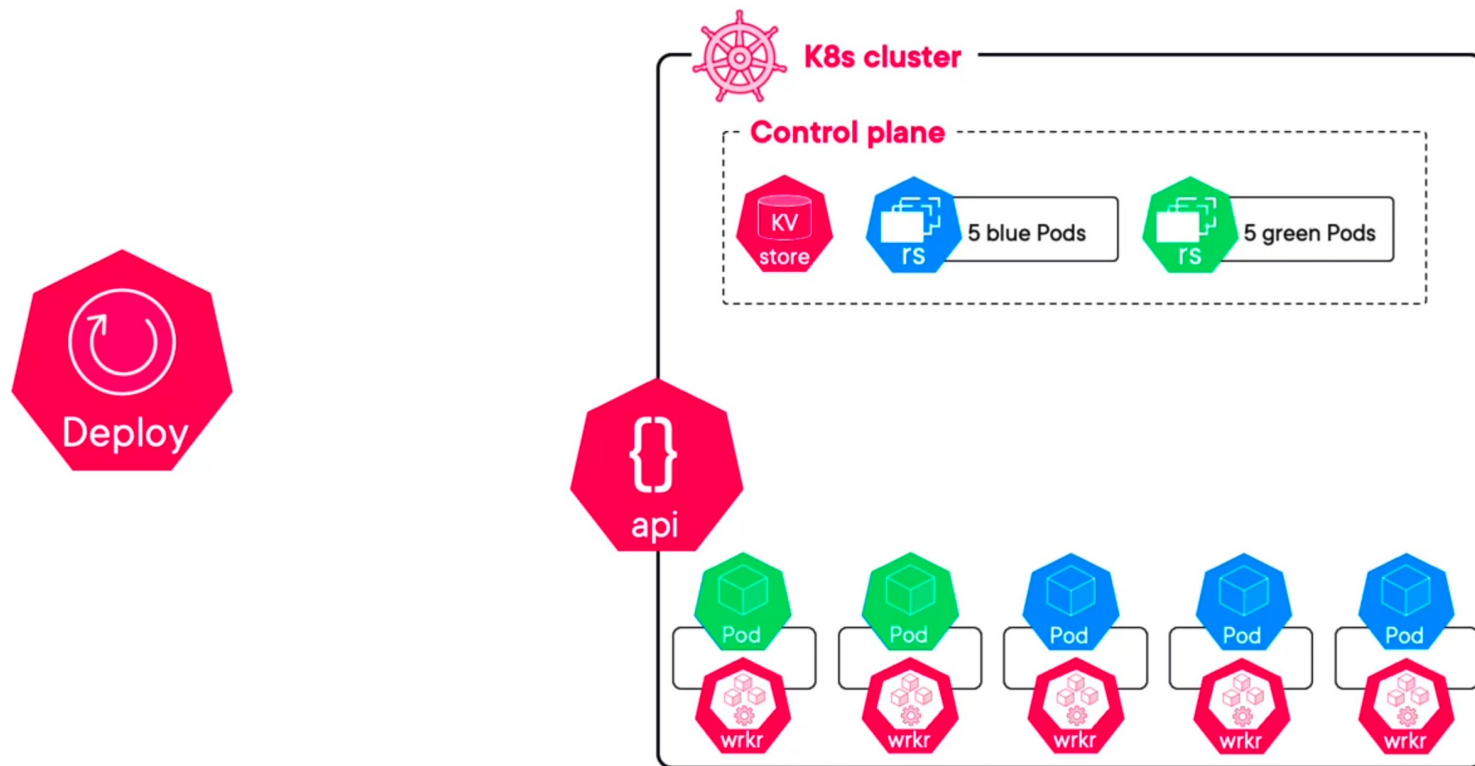
Deployment – Mises à jour en continu (Rolling update)



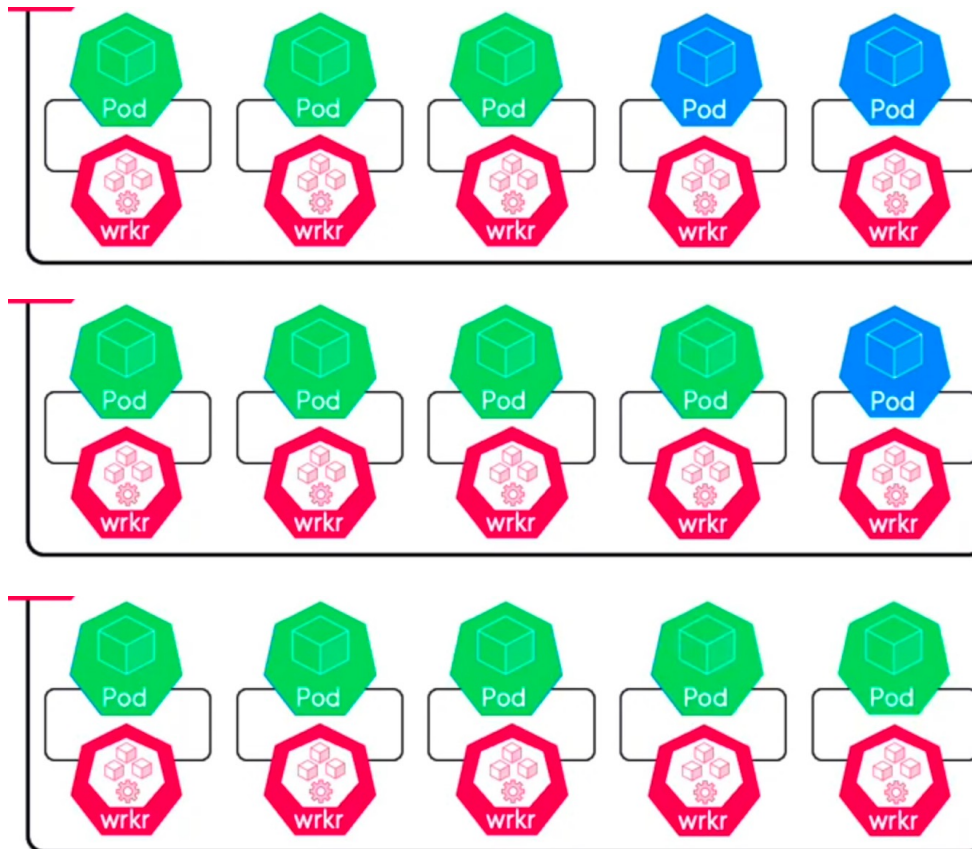
Deployment – Mises à jour en continu (Rolling update)



Deployment – Mises à jour en continu (Rolling update)

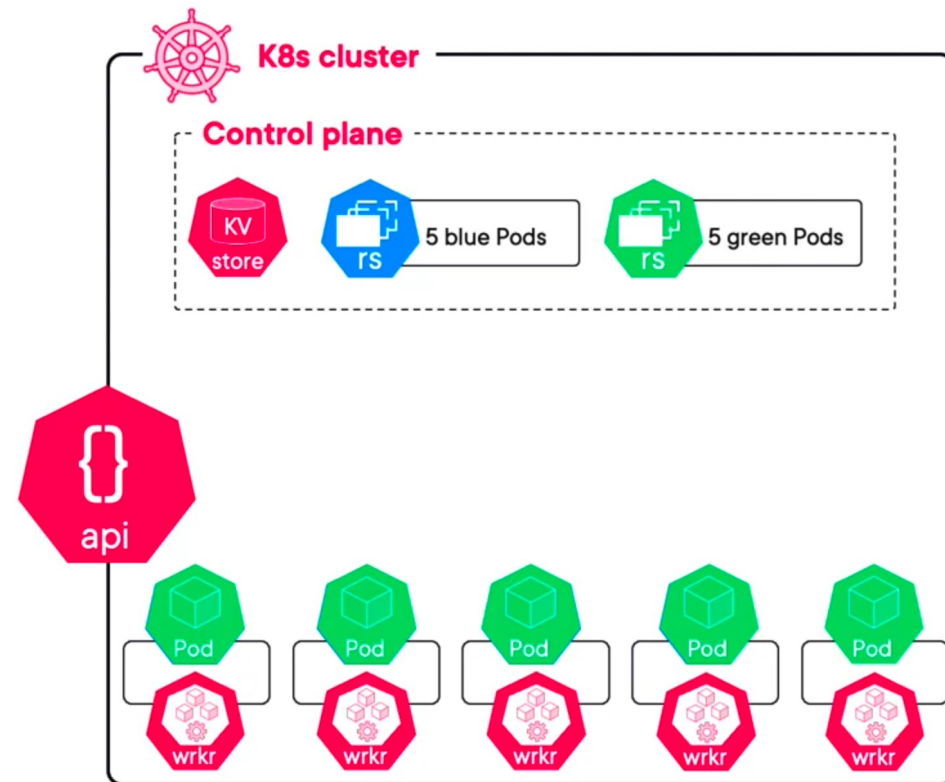


Deployment – Mises à jour en continu (Rolling update)

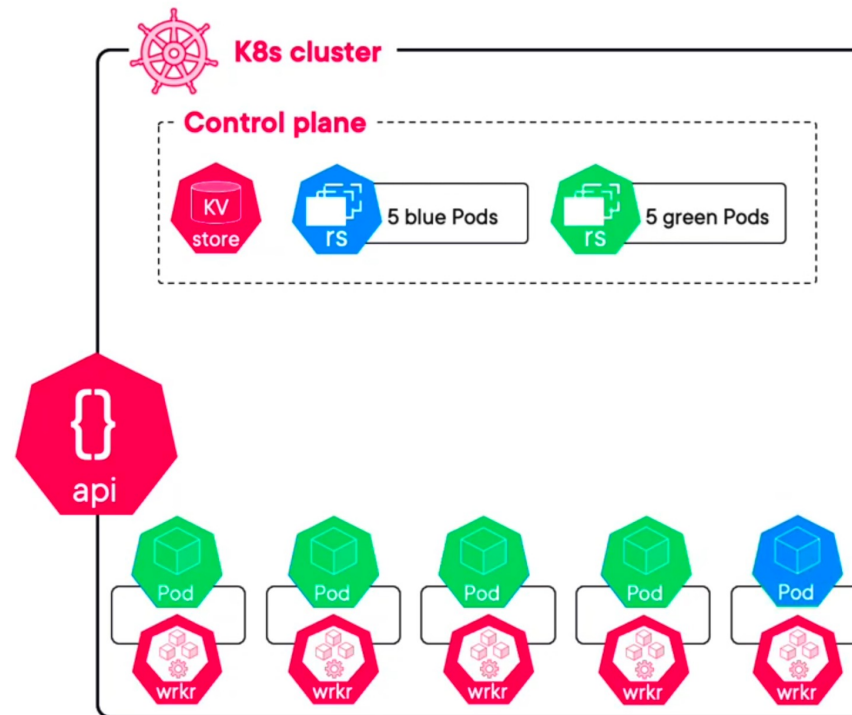


Nouvel état

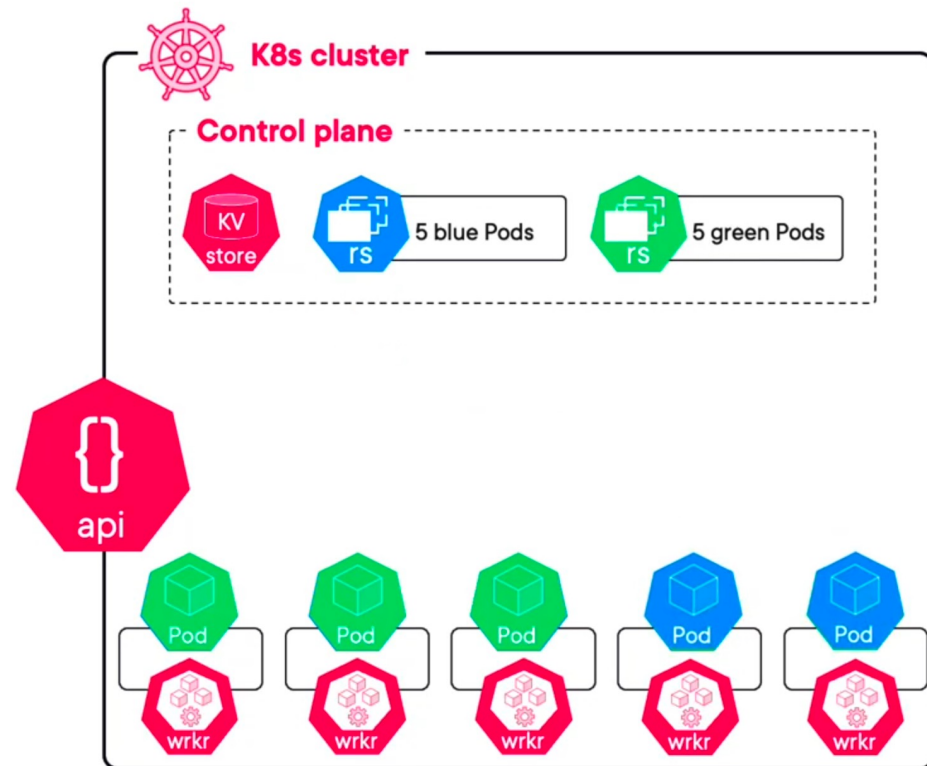
Deployment – Revenir à des versions antérieures(Rollbacks)



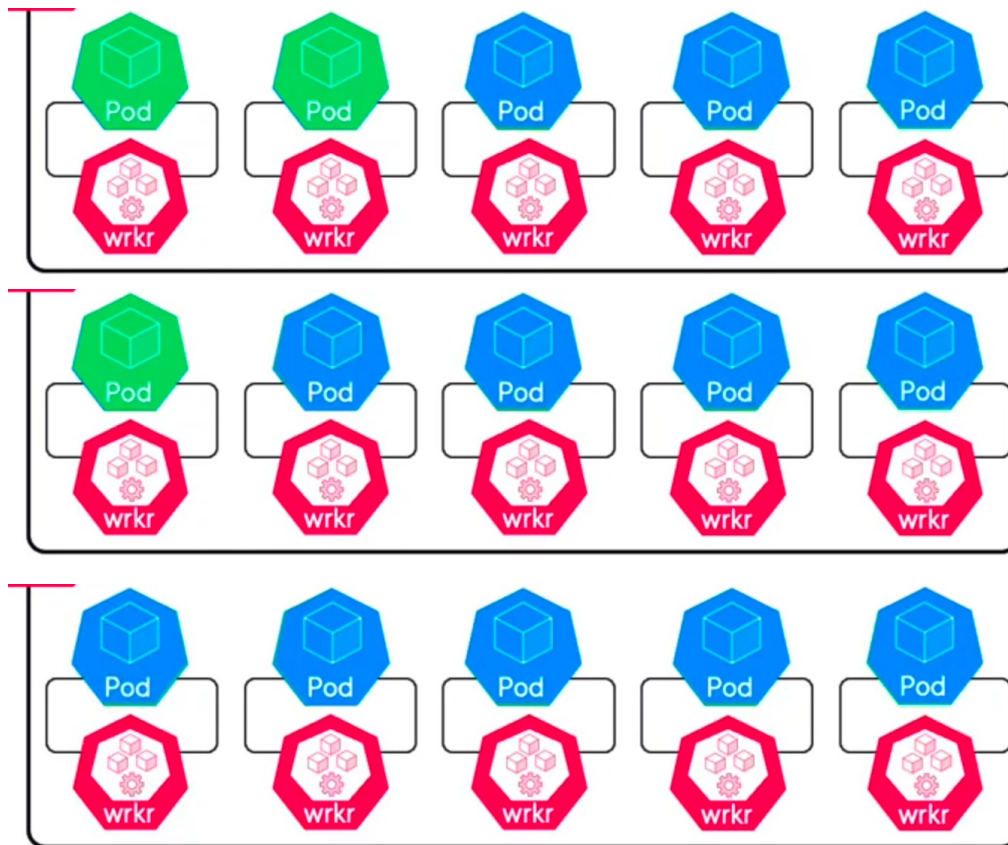
Deployment – Revenir à des versions antérieures(Rollbacks)



Deployment – Revenir à des versions antérieures(Rollbacks)

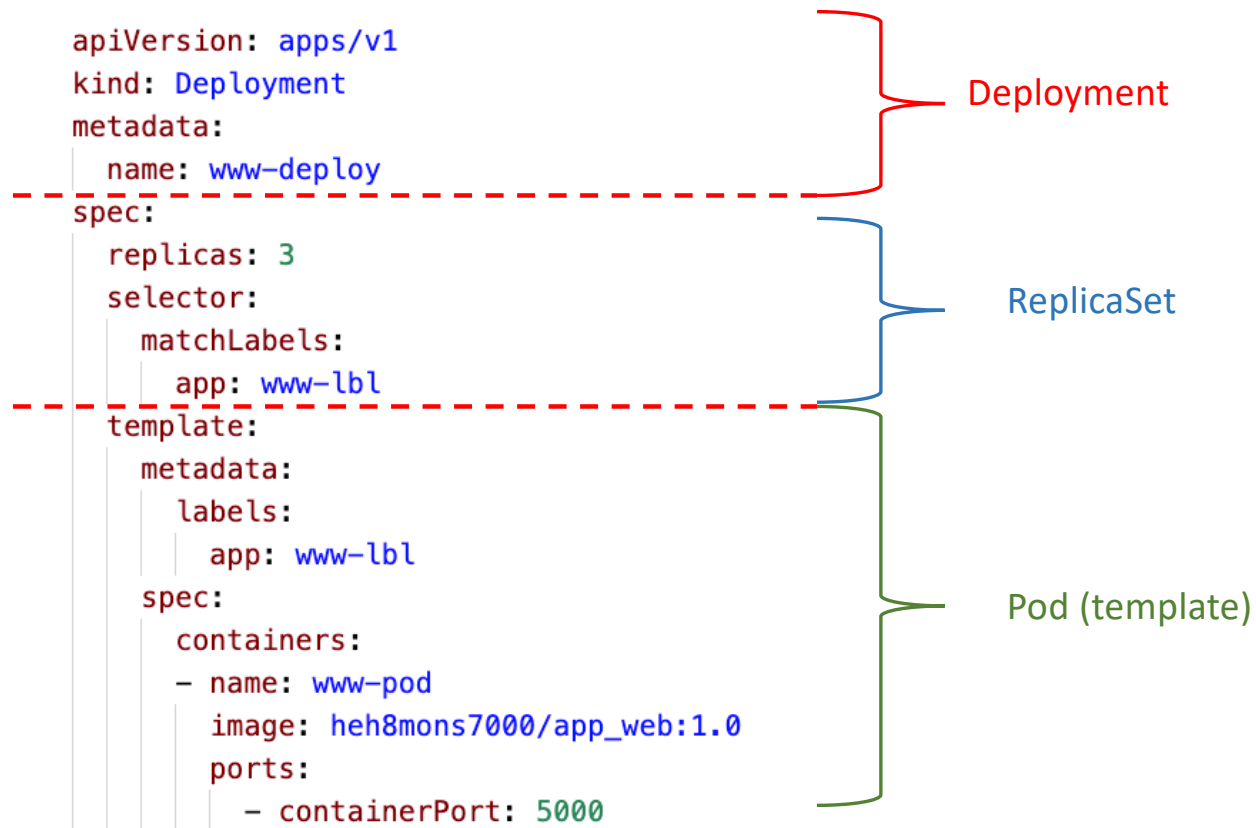


Deployment – Revenir à des versions antérieures(Rollbacks)



Etat antérieur

Spécification – Exemple



Spécification – Exemple

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: www-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: www-lbl
  template:
    metadata:
      labels:
        app: www-lbl
    spec:
      containers:
        - name: www-pod
          image: heh8mons7000/app_web:1.0
          ports:
            - containerPort: 5000
```

Le type de la ressource considérée

Le nom du déploiement

3 répliques seront créées pour ce déploiement

Détermine les Pods qui seront gérés par le **ReplicaSet**.

Seuls les Pods ayant le label « app: www-lbl » seront considérés

Défini le modèle de Pod que le Deployment va gérer

Deployment – Kubectl

```
# Create a Deployment  
kubectl create -f deployment.yml
```

Liste tous les Deployments

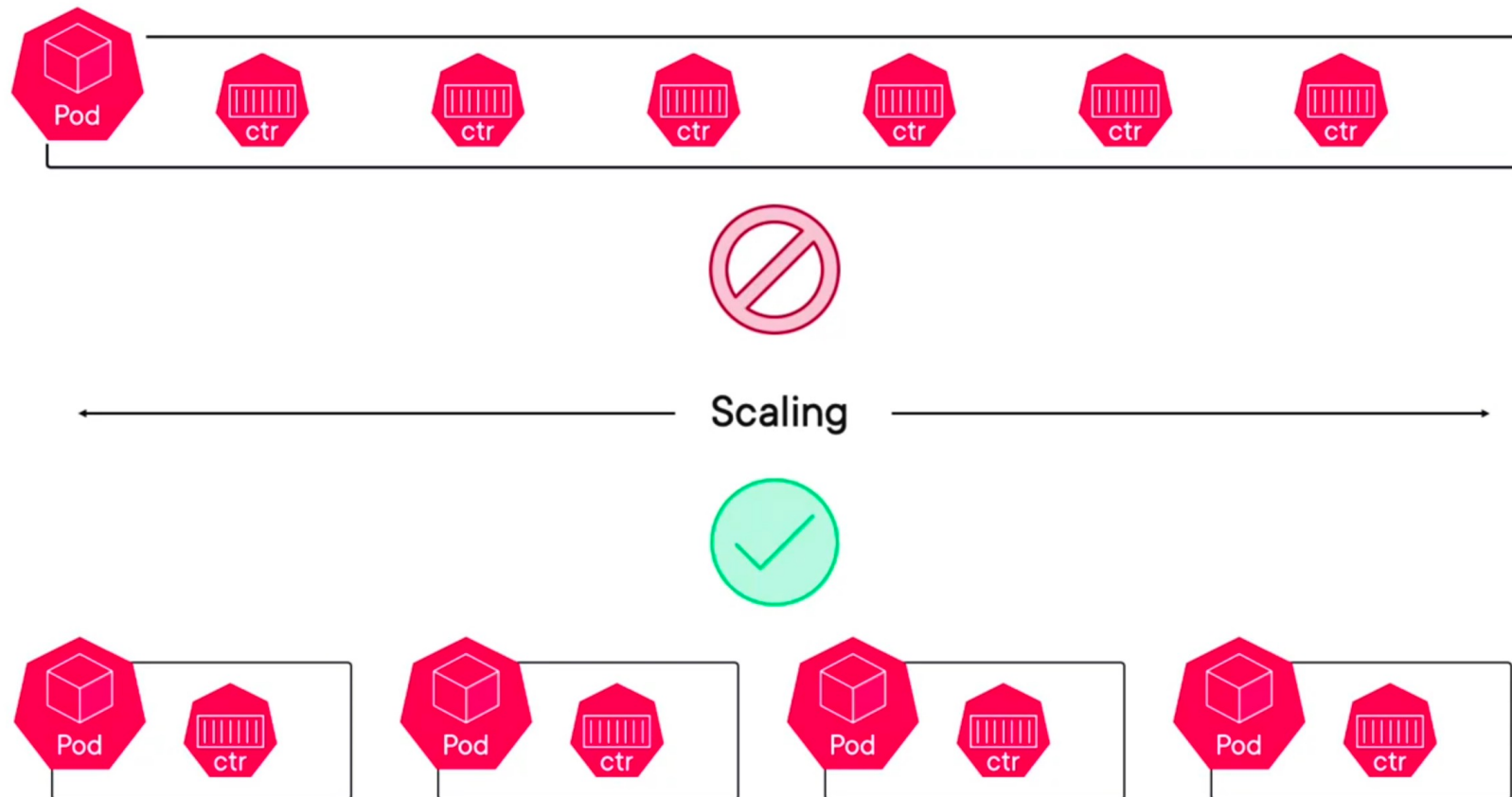
```
kubectl get deployments
```

```
# Get all Deployments with a specific label  
kubectl get deployment -l app=nginx
```

```
# Delete Deployment  
kubectl delete deployment [deployment-name]
```

```
# Scale the Deployment Pods to 5  
kubectl scale deployment [deployment-name] --replicas=5
```

Architecture - Pod



Deployment – Kubectl

Mis à jour continu (rolling update)

```
image: nigelpoulton/k8sbook:edge # This line changed
```

```
Kubectl apply -f deployment.yml --record=true
```

Flag : --record=true (déprécié) -> --revision

=> Enregistrement des changements effectués dans chaque révision

Afficher l'historique des déploiements

```
kubectl rollout history deployment hello-deploy
```

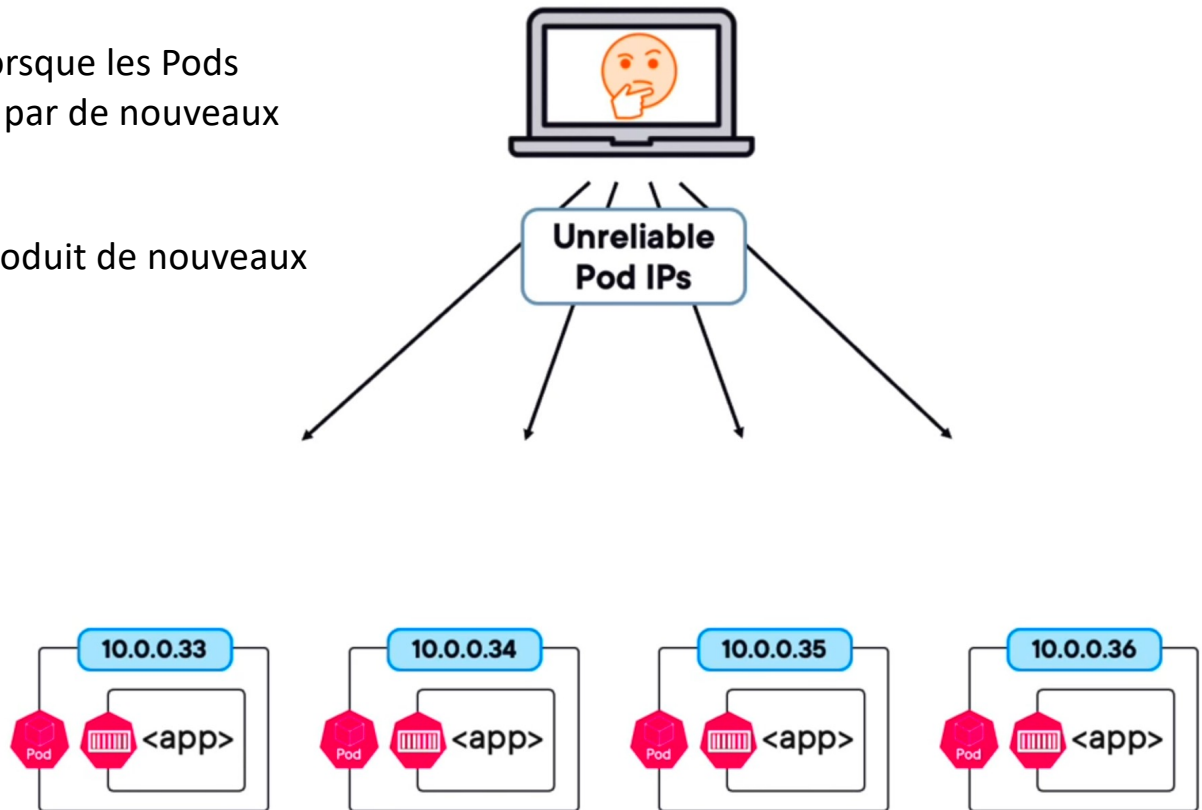
Revenir à une révision antérieure

```
Kubectl rollout undo deployment/hello-deploy
```

```
Kubectl rollout undo deployment/hello-deploy --to-revision=1
```

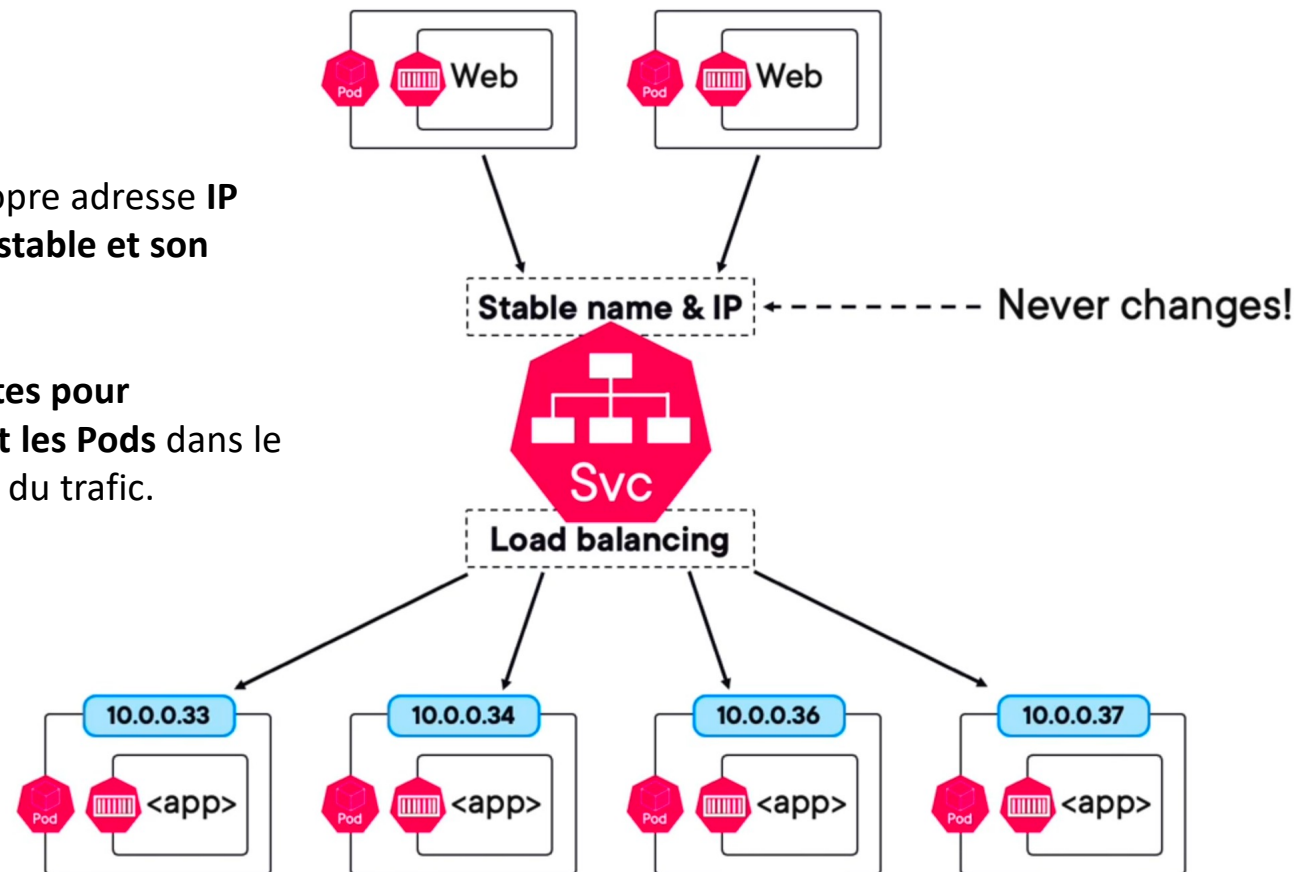
Service – Introduction

- Les **IP des pods ne sont pas fiables**. Lorsque les Pods tombent en panne, ils sont remplacés par de nouveaux Pods qui ont de nouvelles adresses IP.
- L'augmentation d'un déploiement introduit de nouveaux Pods avec de nouvelles adresses IP.

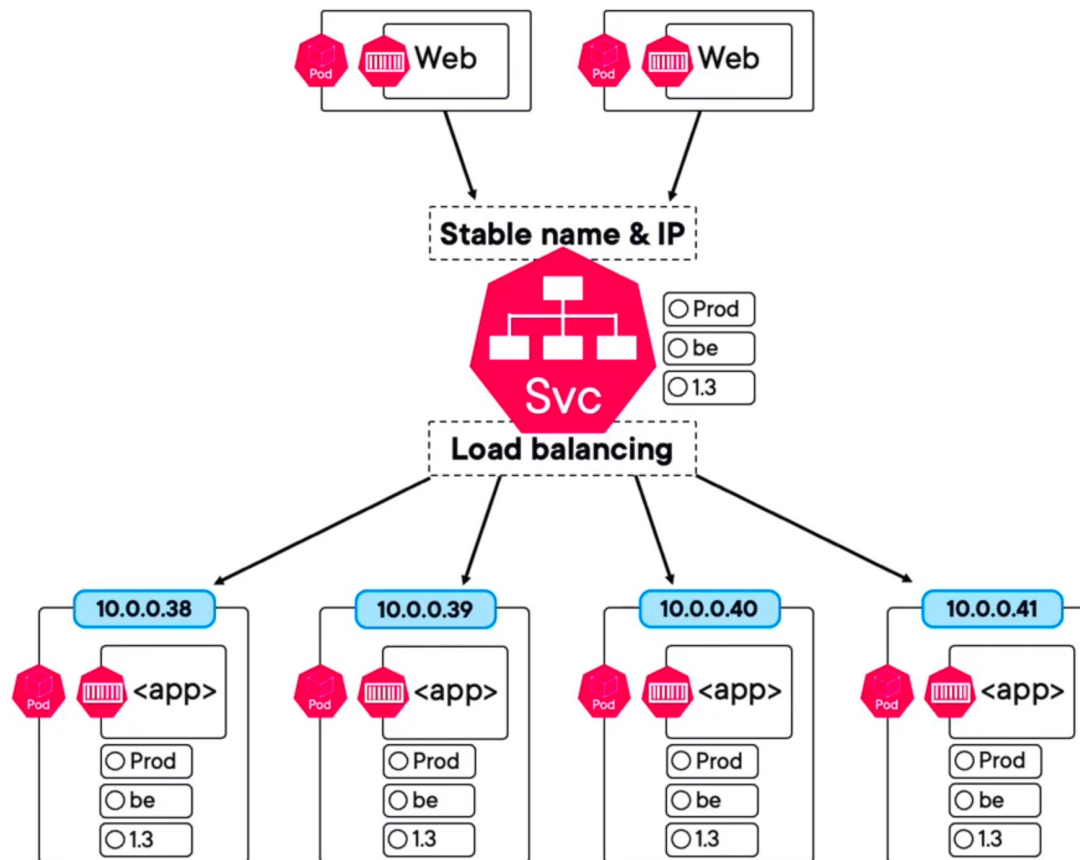


Service – Introduction

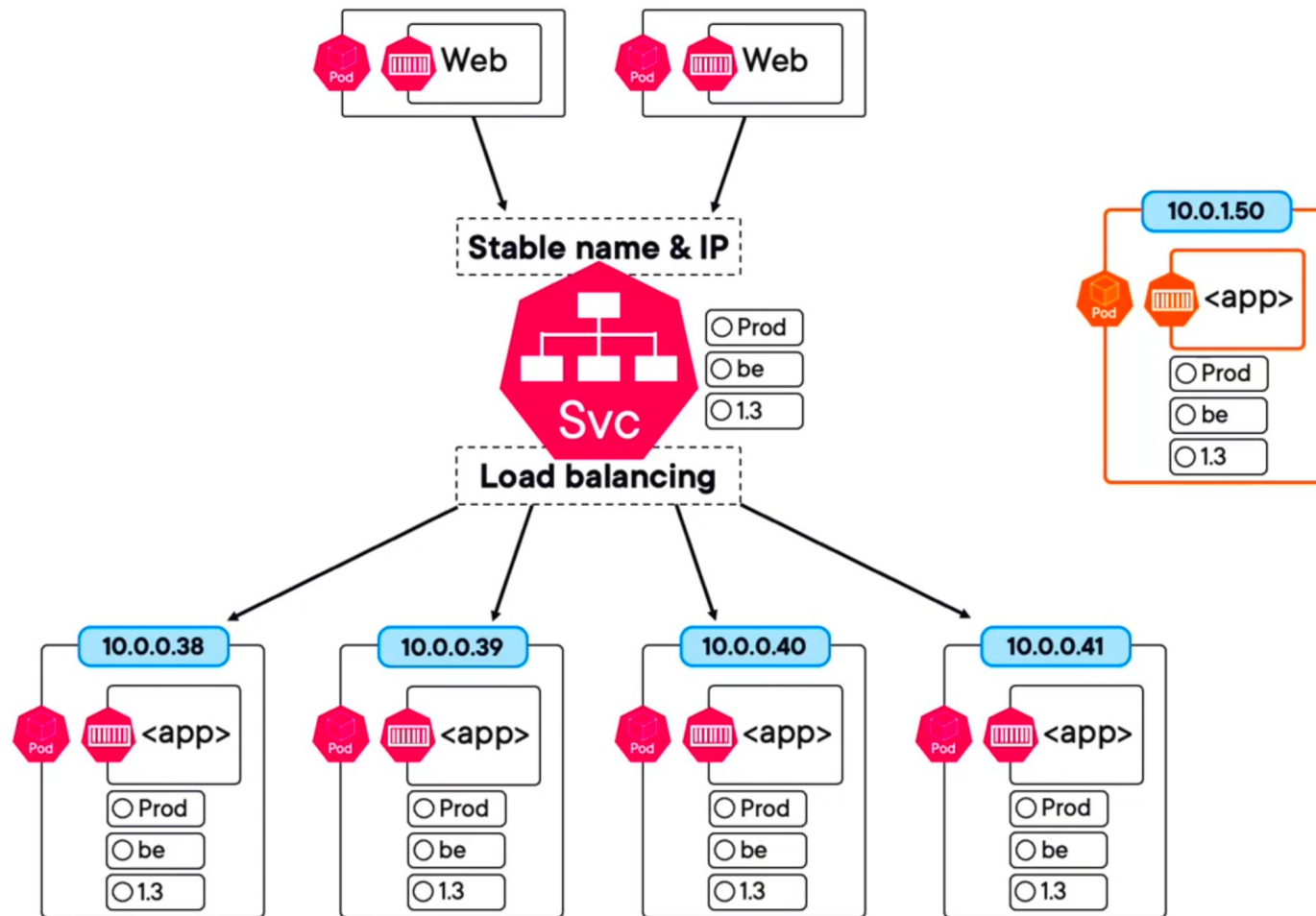
- chaque Service obtient sa propre adresse IP **stable**, son propre nom DNS **stable** et son propre port **stable**.
- Services utilisent des **étiquettes pour sélectionner dynamiquement les Pods** dans le cluster auxquels ils enverront du trafic.



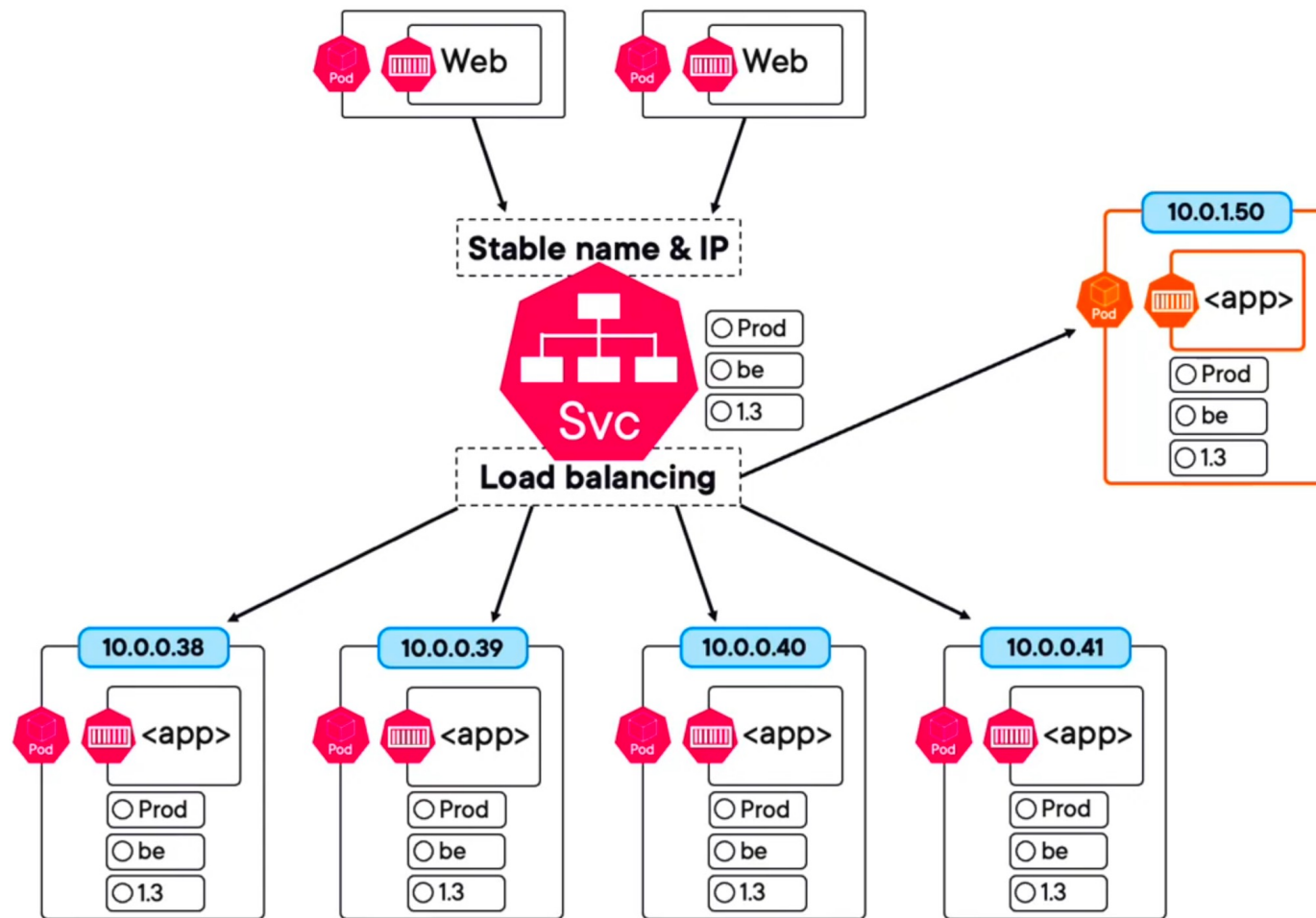
Service – Label



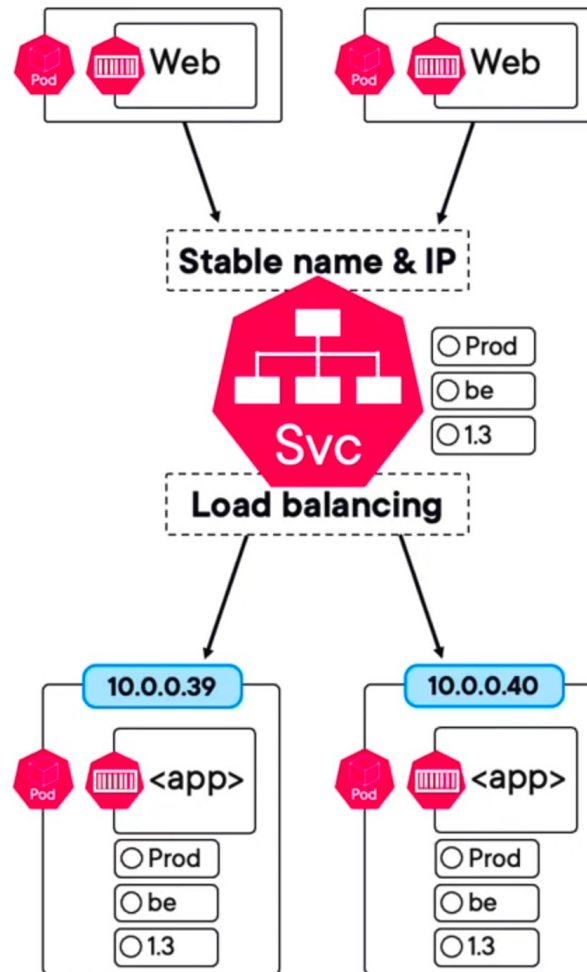
Service – Label



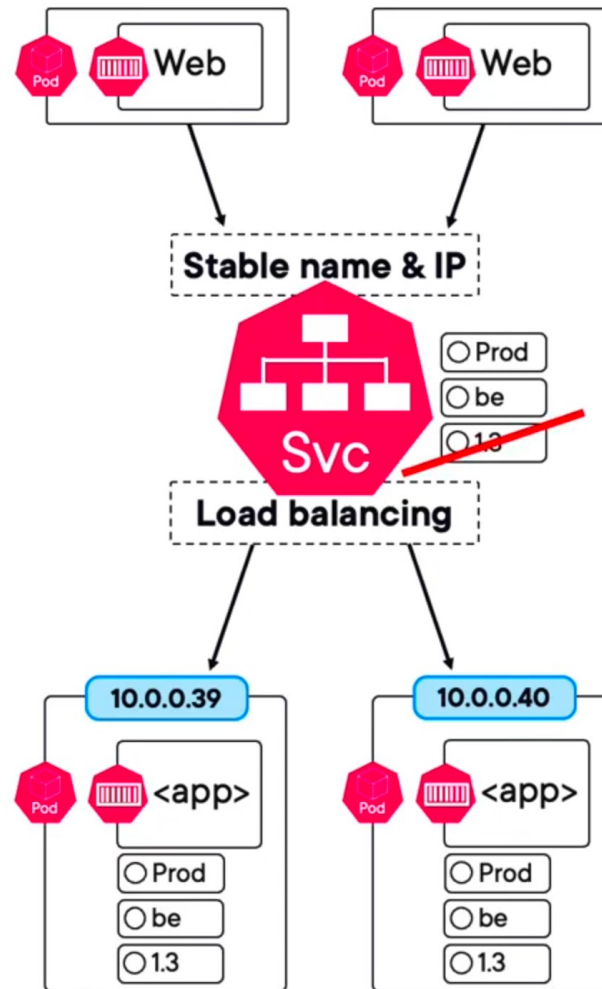
Service – Label



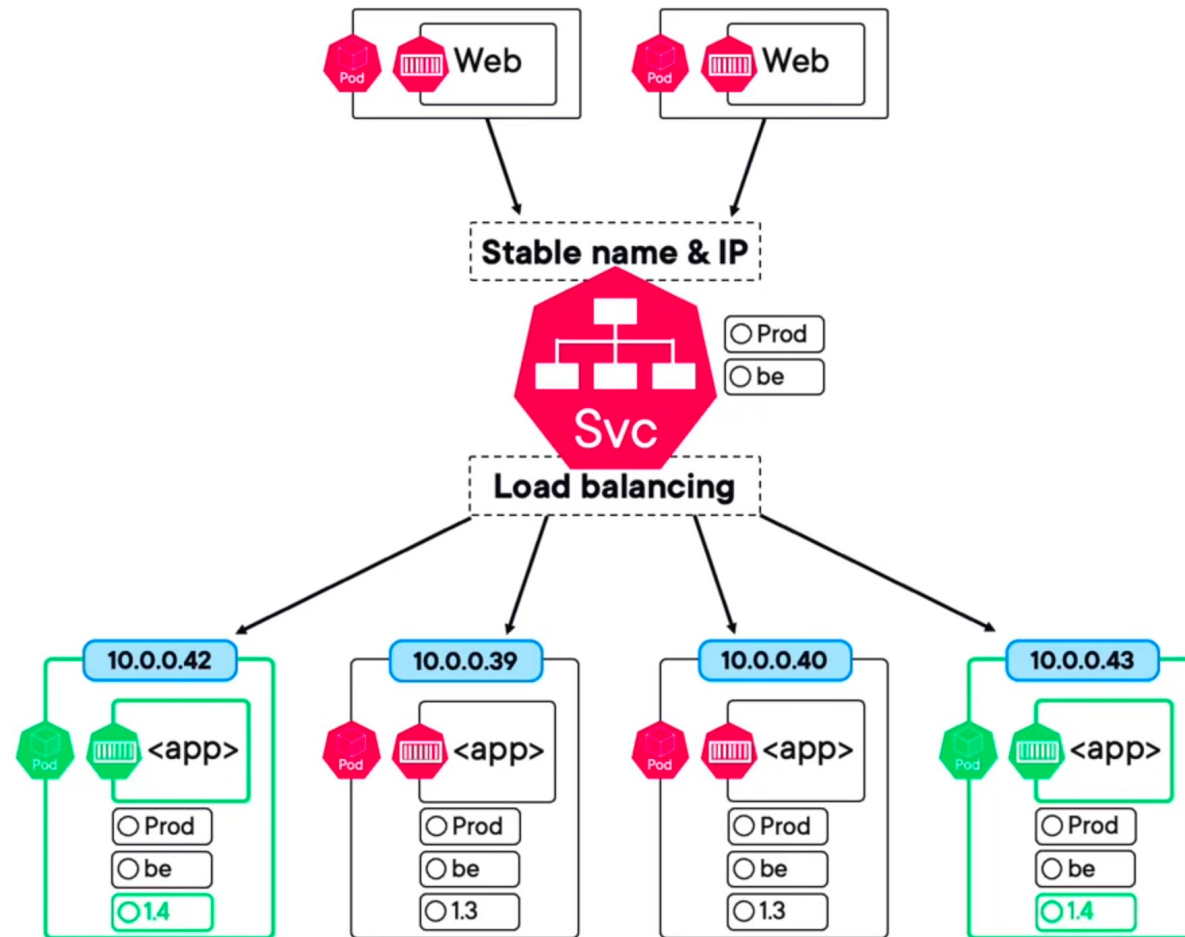
Service – Label



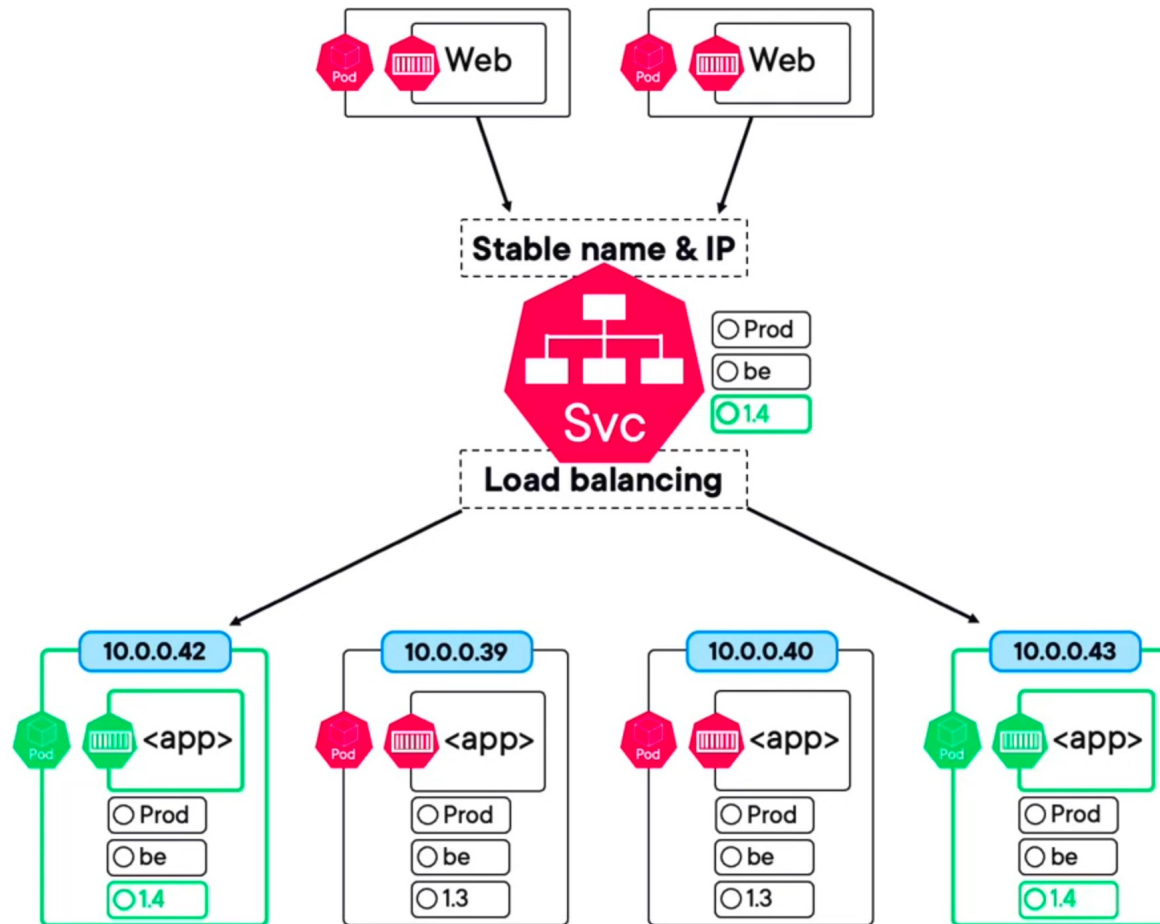
Service – Label



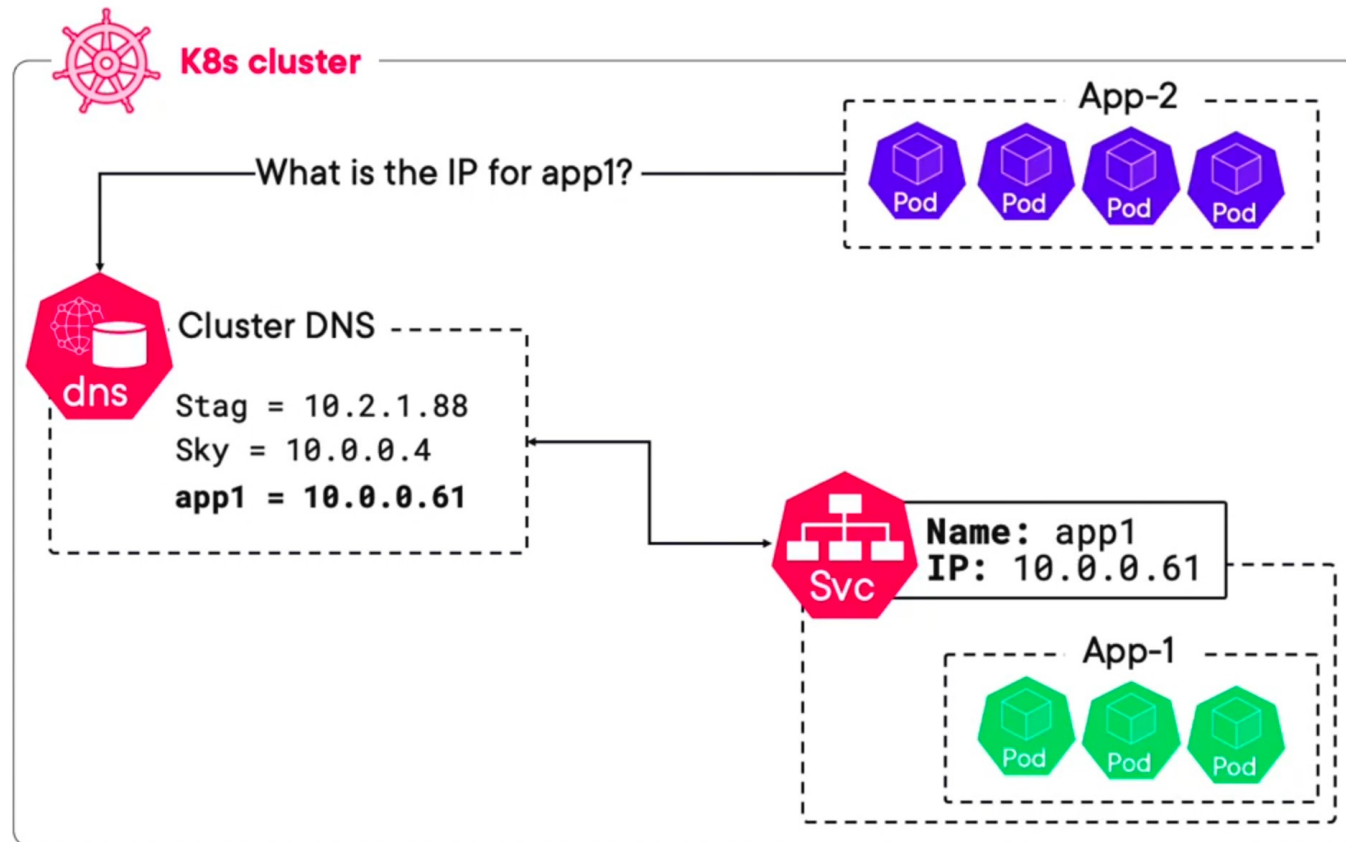
Service – Label



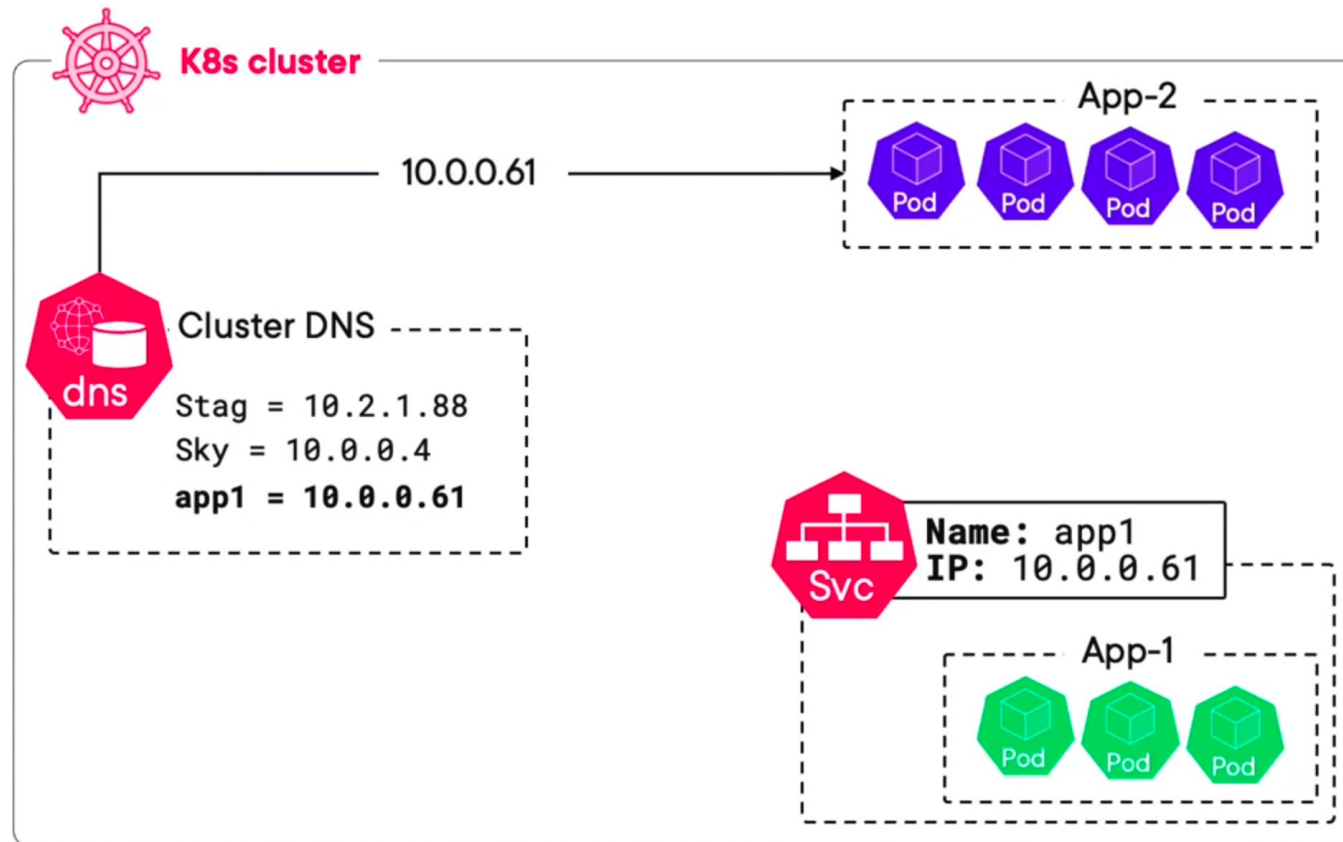
Service – Label



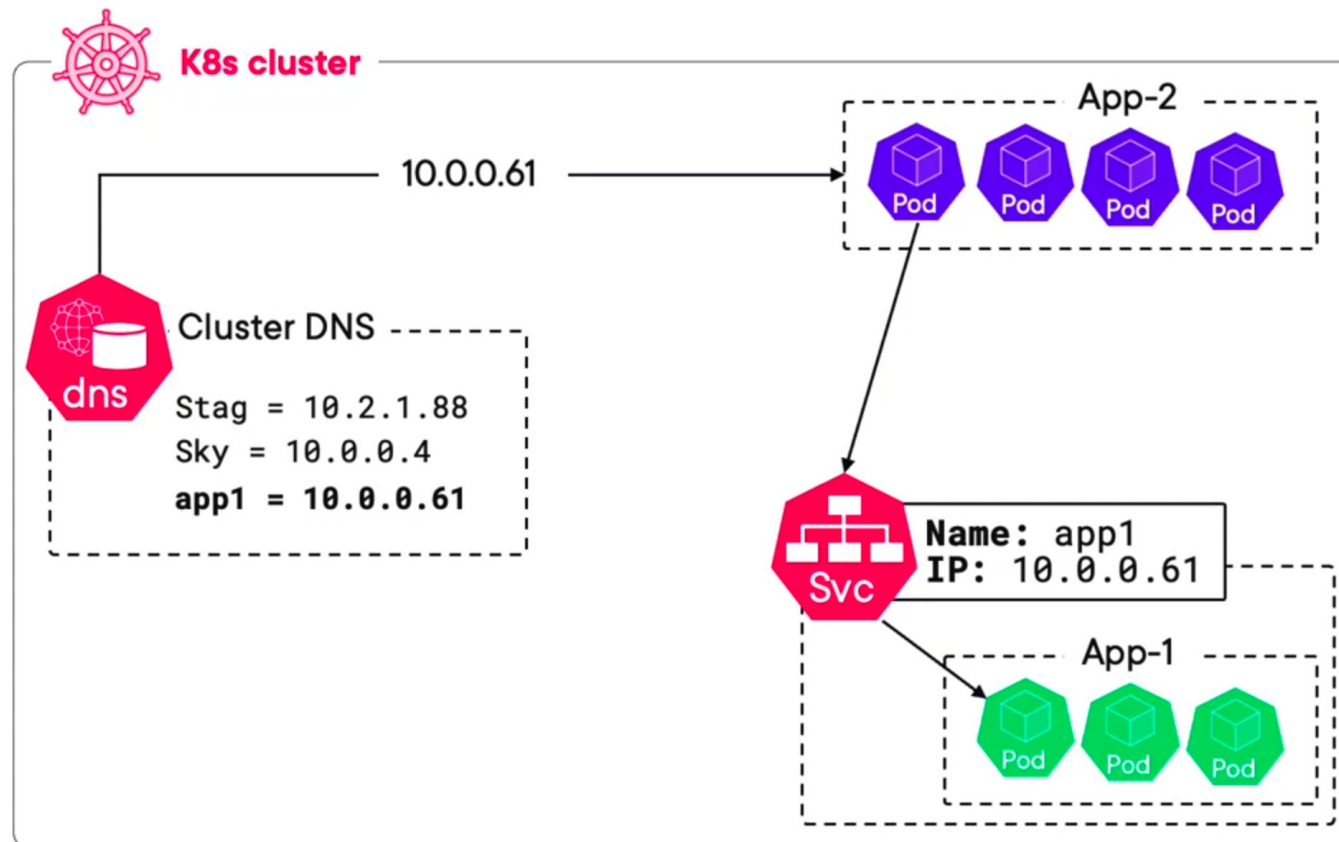
Service – Cluster DNS



Service – Cluster DNS



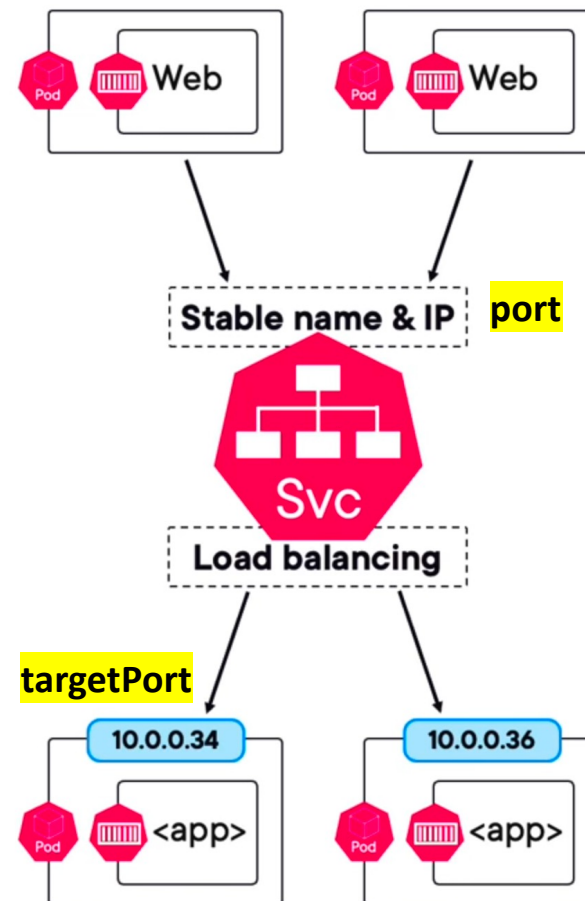
Service – Cluster DNS



Service – Types de service (ClusterIP)

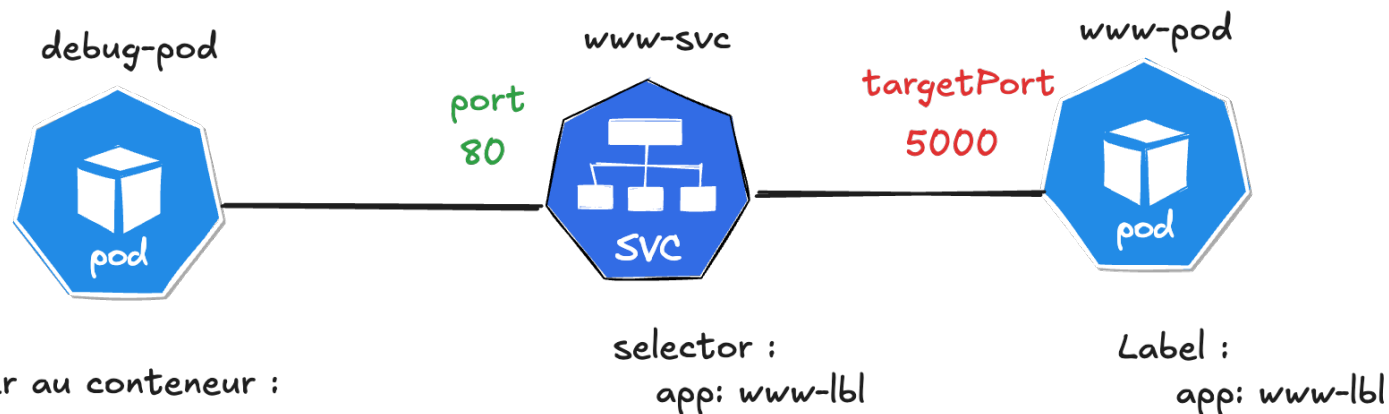
ClusterIP (par défaut) : exposition à l'intérieur du cluster via SERVICE_NAME, ne rend pas le service disponible à l'extérieur du cluster.

ClusterIP (default)
Internal cluster connectivity



Service – Types de service (ClusterIP)

Exemple :



Accéder au conteneur :

```
# kubectl exec -it debug-pod -- sh
```

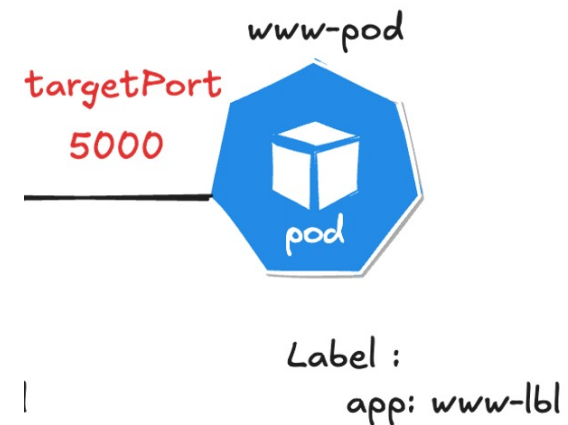
```
# apk add curl
```

tester la communication :

```
# curl http://www-svc
```

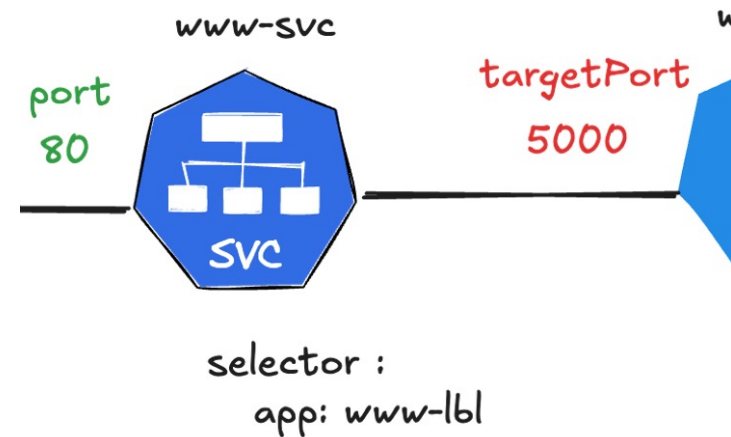
Service – Types de service (ClusterIP)

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: www-pod
5    labels:
6      app: www-lbl
7  spec:
8    containers:
9      - name: www-ctr
10      image: heh8mons7000/app_web:1.0
11      ports:
12      - containerPort: 5000
```



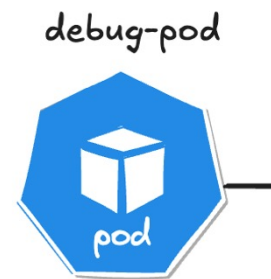
Service – Types de service (ClusterIP)

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: www-svc
5  spec:
6    selector:
7      app: www-lbl
8    type: ClusterIP
9    ports:
10     - port: 80
11       targetPort: 5000
```



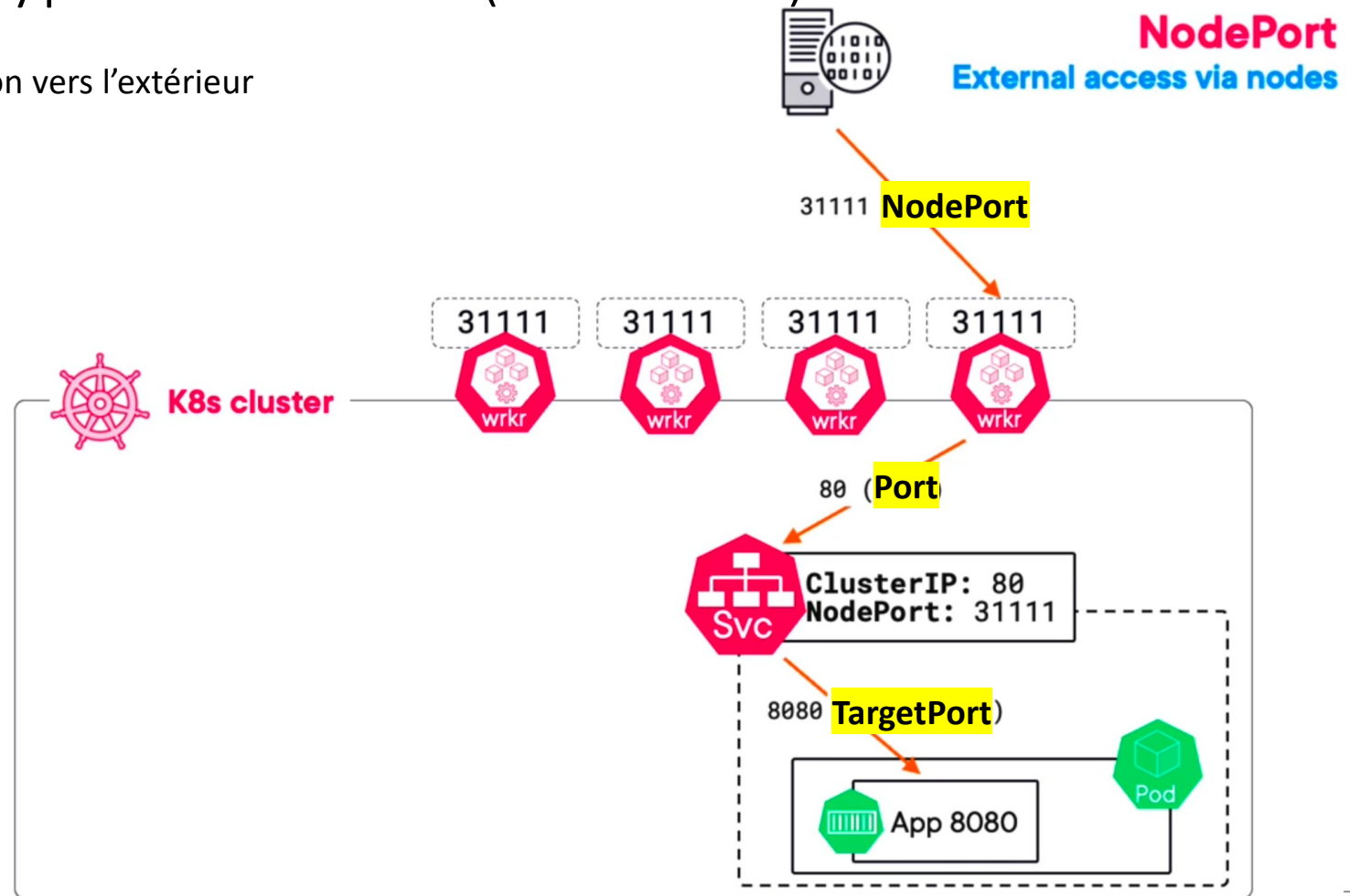
Service – Types de service (ClusterIP)

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: debug-pod
5  spec:
6    containers:
7      - name: debug
8        image: alpine
9        command:
10         - "sleep"
11         - "10000"
```



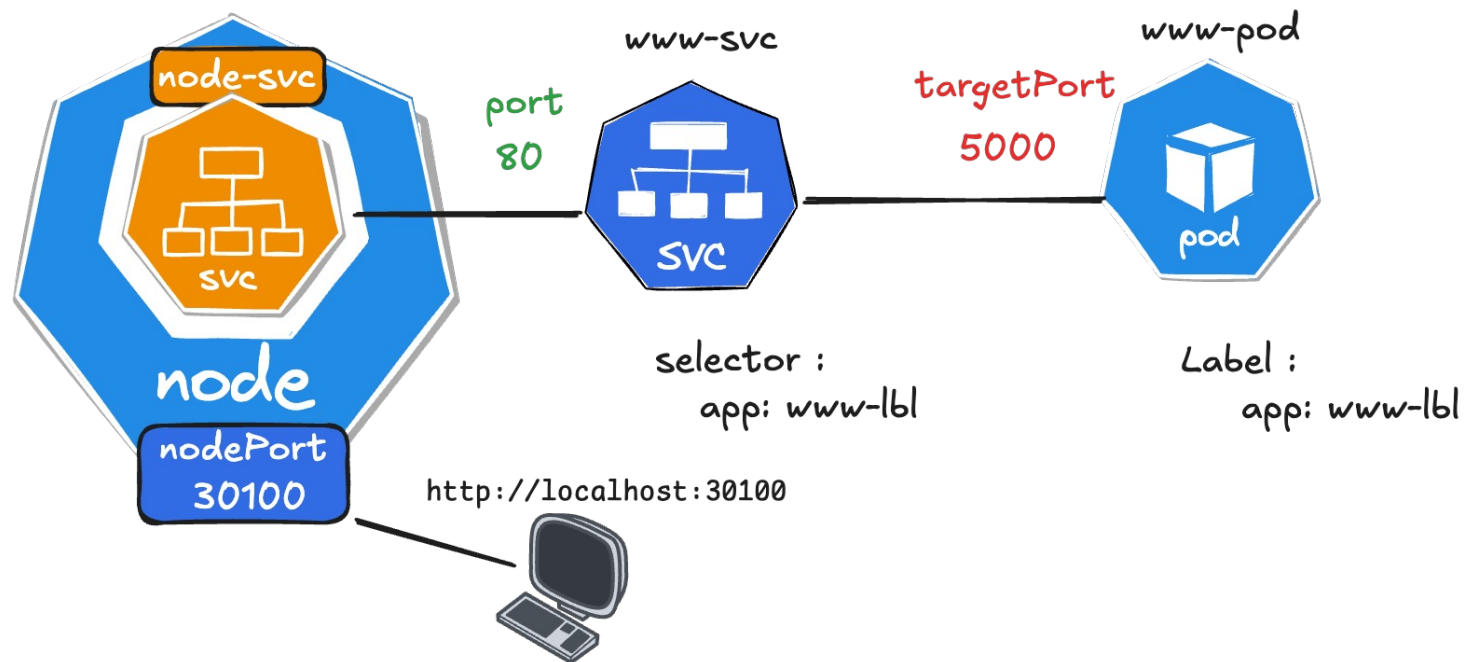
Service – Types de service (NodePort)

NodePort : exposition vers l'extérieur



Service – Types de service (NodePort)

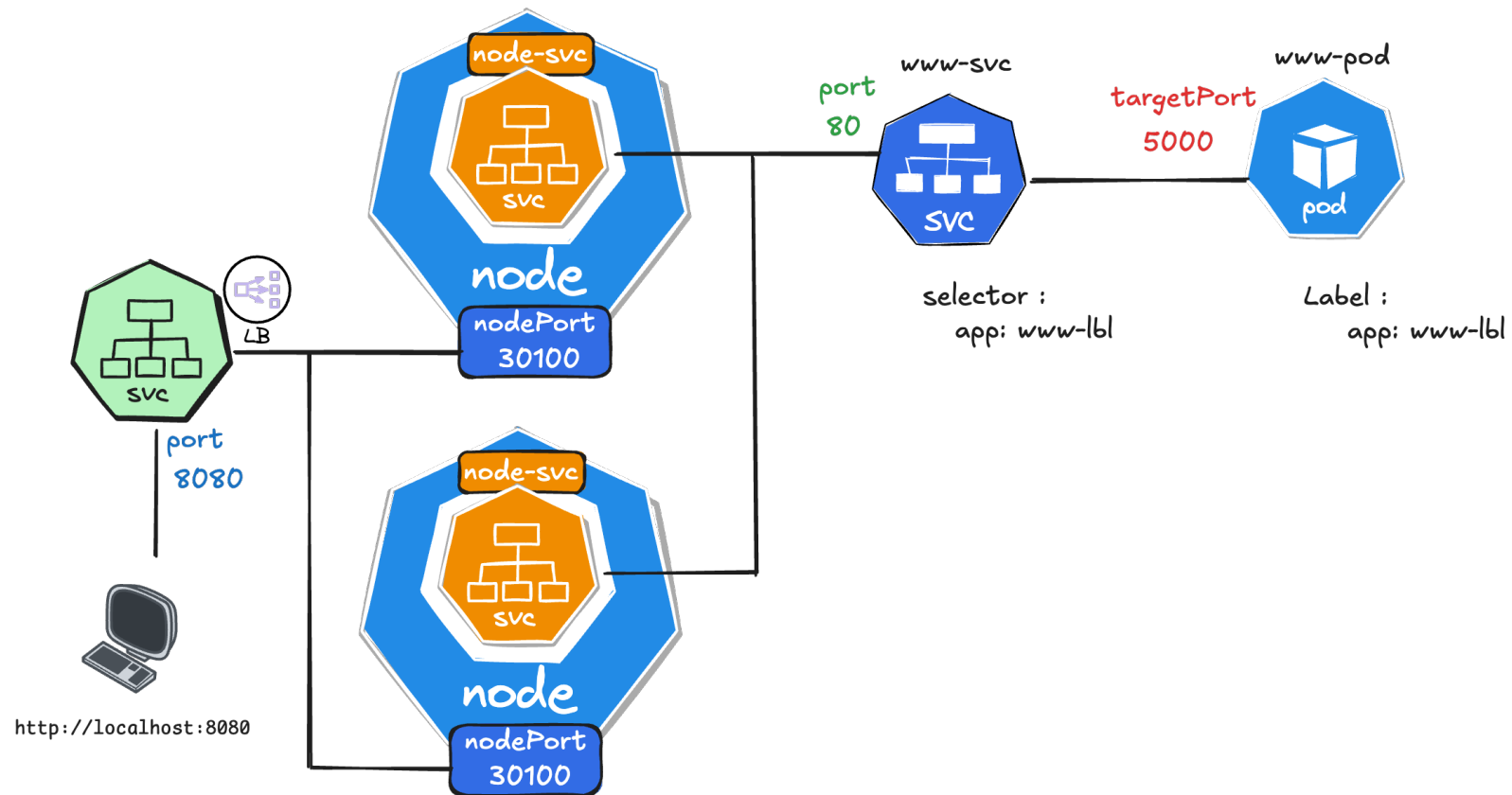
Exemple



Service – Types de service (NodePort)

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: node-svc
5  spec:
6    selector:
7      app: www-lbl
8    type: NodePort
9    ports:
10     - port: 80
11       targetPort: 5000
12       nodePort: 30100
```

Service – Types de service (LoadBalancer)



Service – Types de service (LoadBalancer)

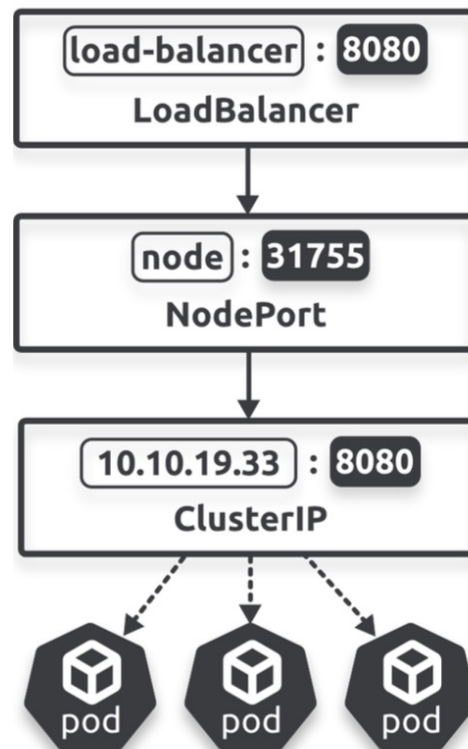
! loadBalancer.yml > ...

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: lb-svc
5  spec:
6    type: LoadBalancer
7    ports:
8      - port: 8080
9        targetPort: 5000
10   selector:
11     app: www-lbl
```

Ce fichier YAML configure un service **LoadBalancer** recevant le trafic sur le port **8080** et le transmettant au port **5000** du container dont les pods sont identifiés par le label **app=www-lbl**.

Les composants **NodePort** et **ClusterIP** requis sont créés automatiquement par Kubernetes.

Service – Empilement de services



Ingress – Définition

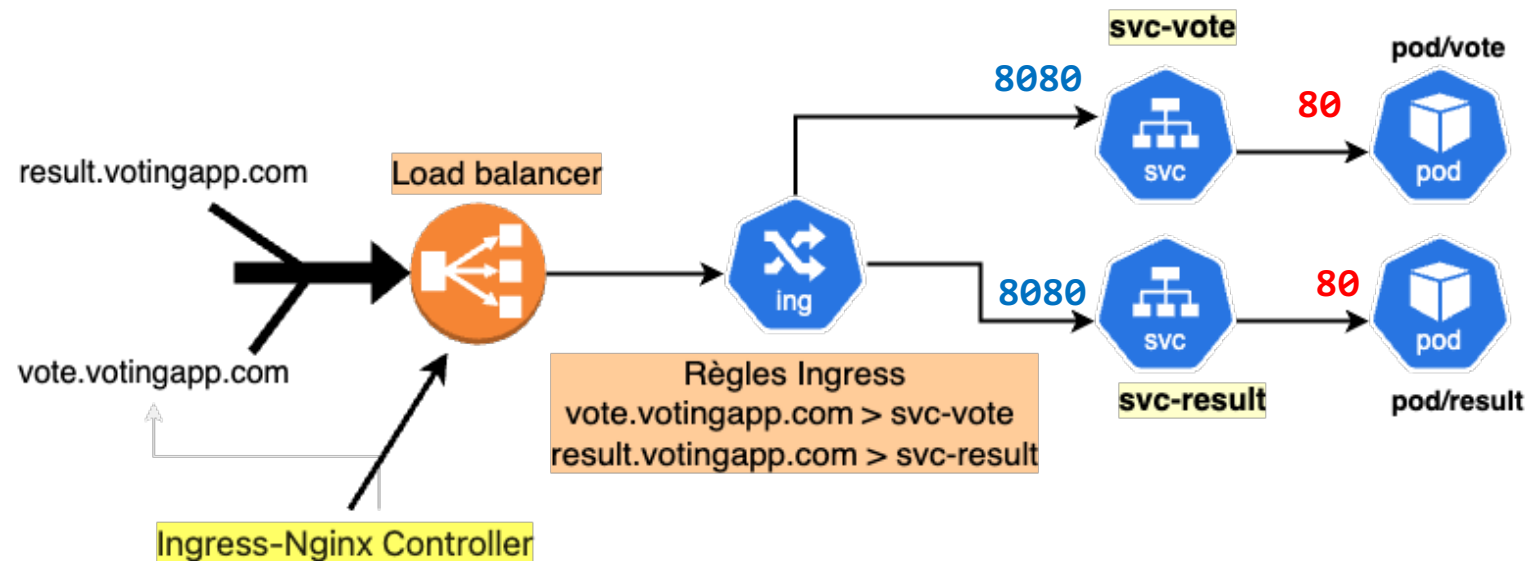
La ressource (Ingress) définit les règles de routage, et le contrôleur(Nginx) les met en œuvre.

Cependant, Kubernetes ne dispose pas d'un contrôleur Ingress intégré, ce qui signifie que vous devez en installer un.

Une fois que vous disposez d'un contrôleur Ingress (Nginx), vous déployez des ressources Ingress avec des règles indiquant au contrôleur comment acheminer les requêtes.

En matière d'acheminement, **Ingress** fonctionne au niveau de la **couche 7 du modèle OSI**, également appelée **couche application**. Cela signifie qu'il peut inspecter les **en-têtes HTTP** et transférer le trafic en fonction des noms d'hôte et des chemins d'accès.

Ingress – Principe



Installation de Ingress-Nginx Controller

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.12.0-beta.0/deploy/static/provider/cloud/deploy.yaml
```

Ingress – Principe

result-svc-clusterip.yaml U ! result-deploymer

gress_vote > ! vote-svc-clusterip.yaml > ...

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    labels:
5      app: vote
6    name: svc-vote
7  spec:
8    type: ClusterIP
9    ports:
10   - port: 8080
11     targetPort: 80
12   selector:
13     app: vote
```

result-svc-clusterip.yaml U × ! result-deploymer

gress_vote > ! result-svc-clusterip.yaml > ...

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    labels:
5      app: result
6    name: svc-result
7  spec:
8    type: ClusterIP
9    ports:
10   - port: 8080
11     targetPort: 80
12   selector:
13     app: result
```

Ingress – Principe

```
! result-svc-clusterip.yaml U    ! result-deployment.yaml M    ! vot
Ingress_vote > ! vote_ingress.yml > ...
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: voting-domain
5  spec:
6    ingressClassName: nginx
7    rules:
8      - host: vote.votingapp.com ← Trafic arrivant via ce nom d'hôte
9        http:
10         paths:
11         - path: /
12           pathType: Prefix
13           backend:
14             service:
15               name: svc-vote ← service ClusterIP « backend » existant appelé « svc-vote »
16               port:
17                 number: 8080 ← Service qui écoute sur le port 8080
18      - host: result.votingapp.com
```

Ingress – Principe

```
18 - host: result.votingapp.com
19   http:
20     paths:
21     - path: /
22       pathType: Prefix
23       backend:
24         service:
25           name: svc-result
26           port:
27             number: 8080
```

Ingress – Enregistrement DNS

Ingress-Nginx Controller => Si installé en local => modifier le fichier hosts

Le fichier est situé :

- sur Mac/Linux -> /etc/hosts
- sur Windows -> C:\Windows\System32\drivers\etc

```
127.0.0.1 vote.votingapp.com  
127.0.0.1 result.votingapp.com
```